COMP3731 Cryptography

Lecture notes

Peter Davies-Peck, Maximilien Gadouleau, and Karl Southern

Academic year 2025-26
Version 2025.11.27

Contents

1 Introduction to cryptography **5**
- 1.1 Fundamentals of (private-key) cryptography . . . 5
- 1.2 Examples of cryptosystems . . . 6
- 1.3 Perfect cryptosystems . . . 7
- 1.4 See further . . . 8

2 Stream ciphers - Theoretical constructions **9**
- 2.1 Semantic security . . . 9
- 2.2 Pseudo-random generators and stream ciphers . . . 10
  - 2.2.1 Security of stream ciphers . . . 10
  - 2.2.2 Issues with stream ciphers . . . 12
  - 2.2.3 Combining PRGs . . . 12

3 Stream ciphers in practice **15**
- 3.1 LFSRs . . . 15
  - 3.1.1 Nonlinearity . . . 16
- 3.2 Cryptosystems . . . 16
  - 3.2.1 Trivium . . . 16
  - 3.2.2 Salsa and ChaCha . . . 17
- 3.3 Generating random bits in practice . . . 17

4 Block ciphers - intro **18**
- 4.1 Definitions and properties . . . 18
  - 4.1.1 Modes of operations . . . 18
  - 4.1.2 Feistel cipher . . . 19
- 4.2 Data Encryption Standard . . . 21
  - 4.2.1 DES . . . 21

5 Attacking block ciphers **26**
- 5.1 Weaknesses of DES . . . 26
- 5.2 Meet-in-the-middle attacks . . . 26
- 5.3 Differential cryptanalysis . . . 27
- 5.4 Linear cryptanalysis . . . 28

6 AES **30**

7 Ideal primitives **32**
- 7.1 Pseudo-random functions . . . 32
- 7.2 Stream ciphers from block ciphers . . . 32
- 7.3 Block ciphers from PRFs and PRGs . . . 34
- 7.4 Ideal Cipher Model . . . 35

8 CPA **36**
8.1 Extending the semantic security game . . . 36
8.1.1 Multi key attacks . . . 36
8.1.2 Chosen plaintext attack . . . 37
8.1.3 IND-CPA. . . . 37
8.2 Modes of operation . . . 37
8.2.1 Cipher Block Chaining . . . 38
8.2.2 Further modes of operation . . . 39

9 Message Integrity **42**
9.1 MAC . . . 42
9.1.1 Constructing a MAC from a PRF - attempt 1 . . . 43
9.2 CBC-MAC - a more efficient approach . . . 45
9.2.1 Extending to arbitrary messages . . . 46
9.3 Further reading . . . 46

10 Hash functions **47**
10.1 Definitions . . . 47
10.2 Security notions . . . 47
10.3 Separations and implications . . . 48
10.4 Further reading . . . 49

11 Hash Constructions I **50**
11.1 Objectives . . . 50
11.2 The Merkle-Damgård transform . . . 50
11.3 Building compression functions . . . 51
11.4 Schemes . . . 52
11.4.1 MD5 . . . 52
11.4.2 SHA-1 . . . 52
11.4.3 SHA-2 . . . 52
11.5 Further reading . . . 52

12 Hash Constructions II and Attacks **53**
12.1 Objectives . . . 53
12.2 Sponge . . . 53
12.2.1 Random Permutation model . . . 54
12.2.2 SHA-3/Keccak . . . 55
12.3 Attacks . . . 55
12.3.1 Time/Space trade off . . . 55
12.4 Further reading . . . 56

13 Random oracles and Message Integrity II **57**
13.1 The Random Oracle Model . . . 57
13.1.1 The random oracle is collision resistant . . . 57
13.2 MACs from Hash functions . . . 58
13.2.1 Hash-and-MAC . . . 58
13.2.2 HMAC . . . 58
13.3 Further reading . . . 59

14 Authenticated Encryption **60**
14.1 Generic constructions . . . 60
14.1.1 Analysis of generic schemes . . . 61
14.2 Secure Communication Sessions . . . 62
14.3 Further reading . . . 62

15 Password Hashing and identification **63**
15.1 Password Hashing 63
15.1.1 PBKDF (Password Based Key Derivation Function) - Slowing down attacks . . 64
15.1.2 MHFs 65
15.2 Multi-factor authentication 66
15.3 Further reading 66

16 KDFs **67**
16.1 KDFs 67
16.1.1 KDFs and random oracles 68
16.2 KDFs in practice 69
16.3 Sources of entropy 69
16.4 Further reading 69

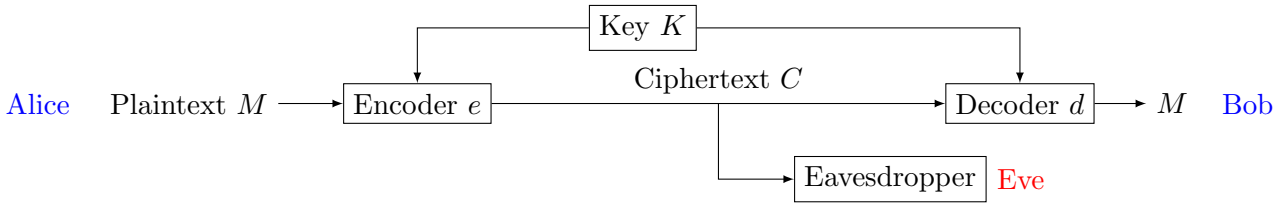
17 Merkle Trees **70**
17.1 Merkle trees 70
17.2 Sorted Merkle tree 71
17.3 Authenticated data-structures 71
17.4 Further reading 71

Lecture 1

Introduction to cryptography

1.1 Fundamentals of (private-key) cryptography

Basic set-up All messages are sent through an external channel:



Now we are interested in protecting the contents of the message from eavesdroppers, known as Eve. The message sender, and encoder, is traditionally called Alice. The receiver, and decoder, is traditionally called Bob.

Notation The message $M \in \mathcal{M}$ is known as the plaintext. Alice and Bob have some secret information $K \in \mathcal{K}$, known as the key. Alice encrypts using an encryption function $e : \mathcal{M} \times \mathcal{K} \to \mathcal{C}$. The transmitted sequence

$$C = e(M, K) \in \mathcal{C}$$

is called the ciphertext. Bob receives C, and decrypts using the decryption function $d : \mathcal{C} \times \mathcal{K} \to \mathcal{M}$. We require that

$$d(e(M, K), K) = M$$

for all plaintexts M. This implies that for a given key K, the encryption function must be injective:

$$e(M_1, K) = e(M_2, K) \implies M_1 = M_2.$$

However, encryption functions do not need to be injective in the key domain, i.e.,

$$e(M, K_1) = e(M, K_2) \not\implies M_1 = M_2.$$

Kerchoffs's principles Clearly, if Eve knows both the decoding algorithm d and the key K, then she can decipher the ciphertext. So at least one of the two should be kept secret. Auguste Kerchoffs argued that only the key should be kept secret:

> The cipher method must not be required to be secret, and it must be able to fall into the hands of the enemy without inconvenience.

This is known as Kerchoffs's principle.

There are three main arguments supporting Kerchoffs's principle:

1. It is much easier to maintain secrecy of a short key than to keep secret the (more complex) algorithm they are using. (Imagine using encryption to secure the communication between all pairs of employees in some organisation.)

2. If the shared, secret information is ever exposed, then it will be much easier to change a key than to replace an encryption scheme.

3. For large-scale deployment, it is much easier for users to all rely on the same encryption algorithm or software (with different keys) than for everyone to use their own custom algorithm. In fact, it is desirable for encryption schemes to be standardise so that compatibility is ensured by default and users will use a scheme that has undergone public scrutiny without weaknesses being found.

Kerchoffs's principle leads us to use cryptosystems whose designs are completely public, instead of trying to rely on "security by obscurity."

Breaking a cryptosystem The enemy (Eve) is assumed to know the encryption function e, the decryption function d, and also to have various additional information, such as language statistics etc. She will certainly have some ciphertext C. All she lacks to decipher to message is the key K.

There are three standard forms of attack:

1. Ciphertext-only attack.

2. Known-plaintext attack. Eve is assumed to also have a sample of plaintext and its encryption.

3. Chosen-plaintext attack. Eve is able to acquire chosen samples of plaintext/ciphertext pairs.

Current cryptosystems are required to be secure against chosen-plaintext attacks.

1.2 Examples of cryptosystems

Early history Aeneas Tacticus has a chapter "On secret messages" in his classic military manual on the art of war (c. 360BC). His most secret method was to thread a string through holes in a piece of bone in an order corresponding to the letters of the message.

In the Caesar cipher, allegedly used by Julius Caesar, involves shifting all the letters of the alphabet a fixed number of letters later in the alphabet (wrapping from z round to a). For instance, shifting by four characters, "`pinkfloyd`" becomes "`tmrojpsch`".

Substitution cipher In the substitution cipher, the key is a permutation of the letters of the alphabet:

$$\begin{aligned} &\texttt{ABCDEFGHIJKLMNOPQRSTUVWXYZ} \\ K = &\texttt{QWERTYUIOPASDFGHJKLZXCVBNM} \end{aligned}$$

Encryption is achieved by applying the permutation letter-wise. For instance, using the key above,

$$\begin{aligned} M &= \texttt{SENDMORETROOPS} \\ C &= \texttt{LTFRDGKTZKGGHL} \end{aligned}$$

This scheme is highly vulnerable to frequency analysis for large texts. As we saw earlier, the letter frequencies are heavily biased in text. Since the substitution cipher preserves the character frequencies, the attacker can guess the key by analysing the frequencies in the text. For instance, the most frequent letter in the ciphertext is probably the image of `E` under the permutation, and so on.

Vigenère cipher The Vigenère cipher is a polyalphabetic substitution cipher, an extension of the Caesar and (simple) substitution ciphers. The key is a sequence of letters, e.g. $K = \texttt{BOTTLEOFRUM}$ The key is repeated until it is as long as the message, and corresponding message and key characters are added (mod 26).

$$\begin{aligned} M &= \texttt{THETREASUREISUNDERTHEROCKBYTHETHIRDPALMTREE} \\ K &= \texttt{BOTTLEOFRUMBOTTLEOFRUMBOTTLEOFRUMBOTTLEOFRU} \\ C &= \texttt{VWYNDJPYMMRKHOHPJGZZZEQREVKYWKLCVTSJUXRIXWZ} \end{aligned}$$

If the length of the key is guess right, then the attacker can still use frequency analysis.

Autokey Vigenère cipher The basic idea is to break the frequency analysis by having a non-repeating key. Again, the key is a sequence of letters e.g. $K = \texttt{BOTTLEOFRUM}$. The key is repeated only once, instead it is augmented to K^* with the start of the message.

$$\begin{aligned} M &= \texttt{THETREASUREISUNDERTHEROCKBYTHETHIRDPALMTREE} \\ K &= \texttt{BOTTLEOFRUM} \\ K^* &= \texttt{BOTTLEOFRUMTHETREASUREISUNDERTHEROCKBYTHETH} \\ C &= \texttt{VWYNDJPYMMRCAZHVJSMCWWXVFPCYZYBMAGGACKGBWYM} \end{aligned}$$

This system was considered unbreakable for nearly 300 years. Then in 1863, a Prussian major discovered that the the length of the keyword can be accurately determined by looking at the separation of repeated digrams. Once the key length is known, frequency analysis can be used to determine the Caesar shift for that set of character positions.

For instance, let us guess a key length of 11. Frequency test each letter of alphabet as candidates for first letter of K. When we hit `B` we decode `T`, use `T` to decode `I`, `I` to get `O`, `O` to get `R`, etc.

1.3 Perfect cryptosystems

Perfect cryptosystems A cryptosystem is perfect if knowledge of the ciphertext C gives absolutely no information about the plaintext M.

Theorem 1.1. *In a perfect cryptosystem there must be at least as many possible keys as possible plaintexts.*

Proof. Suppose $|\mathcal{K}| < |\mathcal{M}|$ and let C be a ciphertext. Let $d(C)$ denote the set of plaintexts that be decrypted from C:

$$d(C) = \{M \in \mathcal{M} : \exists K \in \mathcal{K} e(M, K) = C\}.$$

We then have $d(C) \subseteq \mathcal{M}$. Also, since for every key K encryption is injective, we have $|d(C)| \leq |\mathcal{K}| < |\mathcal{M}|$. Thus, $d(C) \subset \mathcal{M}$ and there exists $M^* \in \mathcal{M} \setminus d(C)$: the ciphertext C gives away the information that the plaintext is not M^*. □

One-time pad In the one-time pad, the key length of the Vigenère cipher is extended to be as long as the plaintext. The key characters are generated uniformly at random.

$$\begin{aligned} M &= \texttt{THETREASUREISUNDERTHEROCKBYTHETHIRDPALMTREE} \\ K &= \texttt{BHMNHRPBHQOILQWDBWPQCJNMYPKIPBZPWFZMDFNRBUC} \\ C &= \texttt{VPRHZWQUCITRELKHGOJYHBCPJRJCXGTXFXDCERALTZH} \end{aligned}$$

Each character of C is now a uniformly random letter, independently of all others. Thus each ciphertext C is equally likely, no matter what the plaintext M is. The One-time pad is a perfect cryptosystem.

There are two main problems with the one-time pad:

1. Key generation. In theory, one would need to generate a very large amount of random characters. Alternatively, one could use a cryptographically secure pseudo-random number generator.

2. Key distribution. The keys are usually not shared amongst only two people, but a whole book of keys is deployed to many agents in the same organisation. This deployment of a large amount of data to many people is not secure in itself.

1.4 See further

Enigma Mechanical cipher machines came into wide use by World War II. The most famous is the German Enigma machine.

Enigma was a polyalphabetic substitution cipher based on electromechanical rotors that changed the substitution cipher after each letter. The substitution cipher would not repeat for 16900 characters. Probably would have been unbreakable if better procedures had been followed during use.

Work on breaking the Enigma cipher led to major advances in computing, in large part by Alan Turing and Gordon Welchman.

Lecture 2

Stream ciphers - Theoretical constructions

2.1 Semantic security

We ended chapter 1 discussing perfect security, which required cryptosystems to have keys that were at least as long as messages, and their impracticality. Instead we will consider a weaker definition called semantic security. With semantic security, rather than requiring that the ciphertext leaks no information about the plaintext, we allow for cryptosystems where the ciphertext gives negligible information about the plaintext. Informally, we consider that if an attacker is given a ciphertext and told that it is the encryption of one of two plaintexts, then they shouldn't be able to work out which it was.

This is formalised by the following game, where our attacker is denoted as the adversary $\mathcal{A}$ and where $b = 0, 1$:

Experiment b

$m_0, m_1 \xleftarrow{\$} \mathcal{A}$

$k \xleftarrow{\$} \mathcal{K}$

$c \xleftarrow{\$} E(k, m_b)$

$\hat{b} \xleftarrow{\$} \mathcal{A}(c)$

Figure 2.1: The experiment for the Semantic Security attack game

Informally, in this game that adversary gets to choose two messages (that are of the same length), they are then encrypted and the adversary is given one of them and has to guess which is was.

For $b = 0, 1$, let W_b be the event that the adversary outputs 1. The adversary's security advantage is defined as:

$$\mathbf{Adv}^{\mathrm{SS}}(\mathcal{A}) := |\Pr[W_0] - \Pr[W - 1]| \ .$$

An equivalent version of this game (often referred to as the bit-guessing version) is

Bit guessing

$m_0, m_1 \xleftarrow{\$} \mathcal{A}$

$b \xleftarrow{\$} \{0, 1\}$

$k \xleftarrow{\$} \mathcal{K}$

$c \xleftarrow{\$} E(k, m_b)$

$\hat{b} \xleftarrow{\$} \mathcal{A}(c)$

Return $b = \hat{b}$

Figure 2.2: The bit-guessing variant of the Semantic Security attack game

Let W be the event that the adversary wins the bit-guessing game, then:

$$\mathbf{Adv}^{\mathrm{SS}}(\mathcal{A}) := 2\left|\Pr[W] - \frac{1}{2}\right| .$$

For a cryptosystem to be semantically secure we require that for all efficient adversaries $\mathcal{A}$ that $\mathbf{Adv}^{\mathrm{SS}}(\mathcal{A})$ is negligible. Where we consider an adversary to be efficient if it is a probabilistic polynomial time algorithm, and $\mathbf{Adv}^{\mathrm{SS}}(\mathcal{A})$ to be negligible if it is $\mathcal{O}(1/f(n))$ for some super-polynomial $f(n)$.

2.2 Pseudo-random generators and stream ciphers

In chapter 1 we looked at the one-time pad. For this to meet the definition of a perfect cryptosystem, it required a key that is at least as long as the message being sent. Ideally we want some method of extending a short key into a much longer one, for use in a one-time pad.

Consider a one-time pad where all messages, ciphertexts and keys are L-bit strings, but we want to use a shorter ℓ-bit seed s as a key. To this we need some deterministic algorithm G that will convert an ℓ-bit string to an L-bit string.

This would give us the following one-time pad, for $m, c = \{0,1\}^L$ and $s = \{0,1\}^\ell$.

$$E(s, m) := G(S) \oplus m$$
$$D(s, c) := G(S) \oplus c$$

This modified one-time pad is a stream cipher and the function G is a pseudo-random generator.

Definition A pseudo-random generator (PRG) is a deterministic algorithm G that takes as input a seed s from a finite seed space $\mathcal{S}$ and outputs r from a finite output space $\mathcal{R}$. Typically $\mathcal{S}, \mathcal{R}$ are sets of bit strings of fixed lengths.

Security For a PRG to be secure, the output should seem random - or given an output from the PRG and a random output an adversary should not be able to distinguish between them. This is formalised by the following security game:

PRG security
$s \xleftarrow{\$} \mathcal{S}$
$r_0 \leftarrow G(s)$
$r_1 \xleftarrow{\$} \mathcal{R}$
$b \xleftarrow{\$} \{0,1\}$
$\hat{b} \leftarrow \mathcal{A}(r_b)$
Return $b = \hat{b}$

Figure 2.3: The PRG Security attack game

Let W be the event that the adversary wins the bit-guessing game, then the adversary's advantage is defined as:

$$\mathbf{Adv}^{\mathrm{PRG}}(\mathcal{A}) := 2\left|\Pr[W] - \frac{1}{2}\right| .$$

A PRG G is secure if for all efficient adversaries $\mathcal{A}$ that $\mathbf{Adv}^{\mathrm{PRG}}(\mathcal{A})$ is negligible.

2.2.1 Security of stream ciphers

We want to be able to prove that if we have a PRG G that is PRG secure, we can build a stream cipher $\mathcal{E}$ that is semantically secure.

Theorem 2.1. *If G is a secure PRG then the stream cipher $\mathcal{E}$ constructed as above is semantically secure.*

In particular, for every adversary $\mathcal{A}$ against the semantic security of $\mathcal{E}$ there exists an adversary $\mathcal{B}$ against the PRG security of G, where $\mathcal{B}$ is an elementary wrapper around $\mathcal{A}$, such that

$$\mathbf{Adv}^{\mathrm{SS}}_{\mathcal{E}}(\mathcal{A}) \leq 2\mathbf{Adv}^{\mathrm{PRG}}_{G}(\mathcal{B}) \ .$$

Proof. We'll prove this using a technique known as game hopping, where we'll start with the original SS game and transform it through a series of "hops" to something that is easier to prove the security of.

First we give Game 0 - the Semantic Security game where the Encryption function is replaced with that of the stream cipher. Let W_0 be the event that $\mathcal{A}$ wins Game 0, then we have $\mathbf{Adv}^{\mathrm{SS}}(\mathcal{A}) = 2\left|\Pr[W_0] - \frac{1}{2}\right|$.

Game 0
$m_0, m_1 \xleftarrow{\$} \mathcal{A}$
$b \xleftarrow{\$} \{0,1\}$
$s \xleftarrow{\$} \mathcal{S}$
$r \leftarrow G(s)$
$c \leftarrow r \oplus m_b$
$\hat{b} \xleftarrow{\$} \mathcal{A}(c)$
Return $b = \hat{b}$

We now adapt Game 0 by replacing $r \leftarrow G(s)$ with $r \xleftarrow{\$} \mathcal{R}$. In effect we're replacing the PRG output with a truly random string.

Game 1
$m_0, m_1 \xleftarrow{\$} \mathcal{A}$
$b \xleftarrow{\$} \{0,1\}$
$s \xleftarrow{\$} \mathcal{S}$
$r \xleftarrow{\$} \mathcal{R}$
$c \leftarrow r \oplus m_b$
$\hat{b} \xleftarrow{\$} \mathcal{A}(c)$
Return $b = \hat{b}$

Game 1 is now depicting a true one-time pad, which is a perfect cryptosystem. Let W_1 be the event that the adversary wins Game 1 (i.e. $b = \hat{b}$). Here $\Pr[W_1] = 1/2$, since we have a perfect cryptosystem an attacker can't distinguish between the two messages.

We now need to relate this to the first game, in particular we want to know how the probability of an adversary winning the game has changed, i.e. $|\Pr[W_0] - \Pr[W_1]|$. In order to do this we will create a new attack $\mathcal{B}$ which is trying to break the PRG security of G that uses $\mathcal{A}$ as a subroutine. $\mathcal{B}$ will receive a string r as input which was either generated using G or was a random string (see fig. 2.3). $\mathcal{B}$ now needs to run $\mathcal{A}$, first it lets $\mathcal{A}$ generate two messages m_0, m_1, then $\mathcal{B}$ chooses a random bit b and computes $c \leftarrow r \oplus m_b$ and sends c to $\mathcal{A}$. $\mathcal{A}$ will then return a bit $\hat{b}$, where it tries to distinguish between which message was encrypted. If $\mathcal{A}$ manages to distinguish correctly then $\mathcal{B}$ will assume that r was generated from G, if $\mathcal{A}$ does not correctly distinguish then $\mathcal{B}$ assumes that r was generated randomly. If r was generated from G then $\mathcal{A}$ is exactly playing Game 0, if r was generated randomly then $\mathcal{A}$ was exactly playing Game 1. From this we get that $|\Pr[W_0] - \Pr[W_1]| = \mathbf{Adv}^{\mathrm{PRG}}(\mathcal{B})$

Combing these together we get

$$\begin{aligned}\mathbf{Adv}^{\mathrm{SS}}(\mathcal{A}) &= 2\left|\Pr[W_0] - \frac{1}{2}\right| \\ &\leq 2\left|\Pr[W_1] + \mathbf{Adv}^{\mathrm{PRG}}(\mathcal{B}) - \frac{1}{2}\right| \\ &= 2\left|\mathbf{Adv}^{\mathrm{PRG}}(\mathcal{B})\right| \ . \quad \square\end{aligned}$$

□

2.2.2 Issues with stream ciphers

Two-time pad We've seen that the one-time pad is secure for a single message, but what if we want to send two messages?

$$\begin{aligned}c_1 &= m_1 \oplus G(s) \\ c_2 &= m_2 \oplus G(s)\end{aligned}$$

If we combine the ciphertexts, we'd have

$$c_1 \oplus c_2 = (m_1 \oplus G(s)) \oplus (m_2 \oplus G(S)) = m_1 \oplus m_2 \ .$$

Unsurprisingly this isn't secure. This highlights the importance of *key reuse* - in general a stream cipher key should never be used multiple times.

Malleability If we allow an eavesdropper to edit messages as well as just reading them, then it is possible for them to change the ciphertext in a predictable way. For example if the attacker wants to change the message by a know amount d, then changing the ciphertext by that amount will do the same to m. If c is the original cipher text and the attack changes it to $c' = c \oplus s$ then:

$$D(s, c') = c' \oplus G(s) = (c \oplus d) \oplus G(s) = m \oplus d \ .$$

Whilst this does require some knowledge on the initial plaintext, this is still an issue.

Are stream ciphers still secure? Whilst both of these are issues, neither are attacks against the Semantic Security game. However they do add to the limitations for when stream ciphers can be used.

2.2.3 Combining PRGs

If we know we have a black box PRG, what do we do if we want to generate more random bits?

Parallel construction Suppose we have a black box PRG G defined over $(\mathcal{S}, \mathcal{R})$, and are confident in generating seeds. We can build G' over $(\mathcal{S}^n, \mathcal{R}^n)$ by running G n times in parallel, i.e. $G'(s_1, \ldots, s_n) := (G(s_1), \ldots, G(s_n))$. Perhaps unsurprisingly the security of this degrades with the number of parallel calls.

Theorem 2.2. *If G is a secure PRG defined over $(\mathcal{S}, \mathcal{R})$ then G' constructed as above is also a secure PRG defined over $(\mathcal{S}^n, \mathcal{R}^n)$. In particular for every adversary $\mathcal{A}$ against the PRG security of G' there exists an adversary $\mathcal{B}$ against the PRG security of G such that,*

$$\mathbf{Adv}^{\mathrm{PRG}}_{G'}(\mathcal{A}) \leq n\mathbf{Adv}^{\mathrm{PRG}}_{G}(\mathcal{B}) \ .$$

The proof of this makes use a proof technique called a hybrid argument. What this does is introduce a series of hybrid games - where Hybrid Game 0 all n outputs are from G, in Hybrid Game 1 the first output is replaced with a random string and the remaining $n-1$ outputs are from G, up to Hybrid Game n where all outputs are replaced with random strings. For each Hybrid Game j, let p_j denote

the probability that $\mathcal{A}$ wins the game (outputs 1). Since Hybrid Game 0 is equivalent to the $b = 0$ case of the PRG security game against G' and Hybrid Game n is quivalent to the $b = 1$ case of the PRG security game against G', we have

$$\mathbf{Adv}^{\mathrm{PRG}}_{G'}(\mathcal{A}) = |p_n - p_0| \ .$$

Then we introduce an adversary $\mathcal{B}$ against the PRG game which receives r and then runs $\mathcal{A}$ as follows:

$\underline{\mathcal{B}(r)}$
$\omega \xleftarrow{\$} \{1, \ldots, n]\}$
$r_1 \xleftarrow{\$} \mathcal{R}$
$\vdots$
$r_{\omega-1} \xleftarrow{\$} \mathcal{R}$
$r_\omega \leftarrow r$
$s_{\omega+1} \xleftarrow{\$} \mathcal{S}$
$r_{\omega+1} \leftarrow G(s_{\omega+1})$
$\vdots$
$s_n \xleftarrow{\$} \mathcal{S}$
$r_n \leftarrow G(s_n)$
send $(r_1, \ldots, r_n)$ to $\mathcal{A}$

where for each fixed $j \in [1, n]$, the $b = 0$ case of $\mathcal{B}$'s attack game is equivalent to Hybrid Game $j - 1$, while the $b = 1$ case is equivalent to Hybrid Game j. Effectively $\mathcal{B}$ is picking a random j and is then running $\mathcal{A}$ on the corresponding Hybrid Game.

Giving (for a fixed j)

$$\Pr[W_0|\omega = j] = p_{j-1} \text{ and } \Pr[W_1|\omega = j] = p_j \ .$$

This gives

$$\Pr[W_0] = \sum_{j=1}^{n} \Pr[W_0|\omega = j] \Pr[\omega = j] = \frac{1}{n} \sum_{j=1}^{n} \Pr[W_0|\omega = j] = \frac{1}{n} \sum_{j=1}^{n} p_{j-1} \ ,$$

and

$$\Pr[W_1] = \sum_{j=1}^{n} \Pr[W_1|\omega = j] \Pr[\omega = j] = \frac{1}{n} \sum_{j=1}^{n} \Pr[W_1|\omega = j] = \frac{1}{n} \sum_{j=1}^{n} p_j \ .$$

N.B The first equality is by the law of total probability, the second equality by the fact that j is chosen uniformly from $[1, n]$.

Finally we have

$$\begin{aligned}
\mathbf{Adv}^{\mathrm{PRG}}_{G} &= |\Pr[W_1] - \Pr[W_0]| \\
&= \left| \frac{1}{n} \sum_{j=1} n p_j - \frac{1}{n} \sum_{j=1}^{n} p_{j-1} \right| \\
&= \frac{1}{n} |p_n - p_0| \\
&= \frac{1}{n} \mathbf{Adv}^{\mathrm{PRG}}_{G'}(\mathcal{A}) \ .
\end{aligned}$$

Sequential construction Suppose we have a black box PRG F defined over $(\mathcal{S}, \mathcal{S} \times \mathcal{R}$ and the ability to transmit a single, but we want to generate more random bits? For ease, we'll say that $\mathcal{S}$ is the set of all s-bit strings and $\mathcal{R}$ is the set of all r-bit strings, giving G as taking s bits as input and

outputs $s+r$ bits. We can chain multiple G's together, by feeding the first s bits of the output of one G to be the seed for the next. Specifically we can construct a new PRG G' defined over $(\mathcal{S}, \mathcal{R}^n \times \mathcal{S})$ as follows:

$\underline{G'(s)}$
$s_0 \leftarrow s$
for $i \leftarrow 1$ to n:
  $(r_i, s_i) \leftarrow G(s_i)$
Return
$(r_1, \ldots, r_n, s_n)$

The security of this degrades linearly with n, similarly to the parallel construction. Interestingly, this means that it's as hard to stretch s bits into $s+1$ random bits as it is to stretch s bits into $s+n$ random bits (for up to polynomial n).

Theorem 2.3. *If G is a secure PRG defined over $(\mathcal{S}, \mathcal{R})$ then G' constructed as above is also a secure PRG defined over $(\mathcal{S}, \mathcal{R}^n \times \mathcal{S})$. In particular for every adversary $\mathcal{A}$ against the PRG security of G' there exists an adversary $\mathcal{B}$ against the PRG security of G such that,*

$$\mathbf{Adv}^{\mathrm{PRG}}_{G'}(\mathcal{A}) \leq n\mathbf{Adv}^{\mathrm{PRG}}_{G}(\mathcal{B}) \ .$$

The proof of this follows a similar hybrid argument to that of eq. (2.2).

Lecture 3

Stream ciphers in practice

3.1 LFSRs

Linear-feedback shift registers(LFSRs) are often used as a building block for constructing a PRG, as they are efficient to implement in hardware and generate outputs that have good statistical properties. An LFSR consists of n memory cells $(s_0, \ldots, s_{n-1})$ each containing 1 bit of data, the bank of all n cells being the register, and a feedback function defined by a set of n boolean feedback coefficients $c_0, \ldots, c_{n-1}$. The feedback function XORs together a subset of the memory cells based on the feedback coefficients. The LFSR starts with an initial state at time $t = 0$, when t is incremented then the value in $s_0^{(t)}$ is outputted. As t increments, all the values in the memory cells are moved down one and $s_{n-1}^{(t+1)}$ is generated using the feedback function. More formally the state at time $t+1$ is defined from time t as:

$$s_i^{(t+1)} \leftarrow s_{i+1}^{(t)}, \qquad i \in [0, \ldots, n-2]$$
$$s_{n-1}^{(t+1)} \leftarrow \bigoplus_{j=0}^{n-1} c_j s_j^{(t)} \ .$$

As the feedback function only generates the new value for s_{n-1}, the first n output bits of the LFSR will be exactly the input.

Example Consider the degree-4 LFSR defined by the coefficients $c_0 = 1, c_1 = 1, c_2 = 0, c_3 = 0$ - i.e as the time increments the value for $s_3^{(t+1)}$ will be $s_0^{(t)} \oplus s_1^{(t)}$. Let the initial state be $(s_3, s_2, s_1, s_0) = (0, 1, 1, 0)$, then the states for each time step will be:

$$\begin{aligned}
t = 0; &\quad s = (0, 1, 1, 0) \\
t = 1; &\quad s = (1, 0, 1, 1) \\
t = 2; &\quad s = (0, 1, 0, 1) \\
t = 3; &\quad s = (1, 0, 1, 0) \\
t = 4; &\quad s = (1, 1, 0, 1) \\
t = 5; &\quad s = (1, 1, 1, 0) \\
t = 6; &\quad s = (1, 1, 1, 1) \\
t = 7; &\quad s = (0, 1, 1, 1) \\
t = 8; &\quad s = (0, 0, 1, 1)
\end{aligned}$$

How long can the output be? A degree-n LFSR has 2^n possible states - corresponding to the possible values of the bits in the register. If the state of the LFSR is ever the all 0 vector, then it will remain in that state forever. If it never hits the all 0 vector, then it will always cycle periodically - if a state repeats then the sequence will repeat, as there is only a finite number of states it must

eventually repeat a state. An LFSR is said to be of maximum length if it cycles through all $2^n - 1$ nonzero states before repeating, this is dependent entirely on the feedback coefficients. It is possible to choose coefficients such that this happens - specifically such that the polynomial defined by the coefficients is primitive - understanding and proving this is beyond the scope of these lectures.

Building a steam cipher Consider the PRG G defined as follows: let L be some predetermined maximum length degree-n LFSR, G takes as input some n bit key k (note this can never be the all 0 key), this is then the initial state of L, G runs L for as many timesteps as need and outputs the 'output' bit from each timestep. Unfortunately this isn't quite secure.

Firstly, lets assume that the coefficients are known (Kerchkhoff's principle) - then the first n bits of the output will reveal the initial state (i.e. the key). If we set G to ignore the first n bits of the output of L, still any n consecutive bits will allow an attacker to reproduce the remaining bits of the output.

Even if extend the key to be $2n$ bits, where the first n bits represent the coefficients, and the remaining n bits form the initial sate, it is possible to extract the coefficients from the first $2n$ bits of the output. The first n output bits $(y_0, \ldots, y_{n-1})$ are the initial state, and then the second n output bits $(y_n, \ldots, y_{2n-1})$ can be used to generate n linear equations with n unknowns (the coefficients) as follows

$$\begin{aligned} y_n &= c_{n-1}y_{n-1} \oplus \ldots \oplus c_0 y_0 \\ &\vdots \\ y_{2n-1} &= c_{n-1}y_{2n-2} \oplus \ldots \oplus c_0 y_{n-1} \ . \end{aligned}$$

If L is a maximum length LFSR then these are linearly independent, and thus can be solved efficiently using linear algebra.

3.1.1 Nonlinearity

In order to address the issue with LFSRs, we need to introduce some element of non-linearity.

Nonlinear feedback function The first option is to replace the linear feedback function with some nonlinear function g. Where the output of g is 0 (or 1) with probability roughly 1/2 over the inputs - this is then known as a FSR. The stream cipher Trivium uses this method.

Nonlinear output Rather than the output of the LFSR being the rightmost bit, the output could be some nonlinear function of the entire register. This method is used by the stream cipher 'snow' that encrypts 3GPP phone traffic.

Nonlinear combiner An alternative is to run several LFSRs in parallel and combine them in a nonlinear fashion. The LFSRs do not need to have the same degree.

Warning! The nonlinear function needs to always be balanced. Consider the nonlinear combiner $a \wedge b \wedge c$, where a, b, c are the outputs of three LFSRs, then this will output 0 approximately 7/8 of the time.

3.2 Cryptosystems

3.2.1 Trivium

Trivium uses three nonlinear FSRs denoted A (which has degree 93), B (which has degree 84) and C (which has degree 111). The output of each FSR is the XOR of the right-most bit and on additional bit from within the register, with the output of Trivium being all three of these outputs XORed together.

The new leftmost bot of each FSR is computed as a function of one bit from the same FSR and a subset of the bits of another FSR - making the feeback function nonlinear. Trivium takes as input an 80-bit key and and 80-bit initialisation vector (IV), the key is loaded into the 80 leftmost bits of A, and the 80 bit IV is loaded into the 80 leftmost bits of B. All remaining bits are set to 0, except for the three rightmost bits of C (which are set to 1). Rather than taking the output from the first tick, Trivium is run for $4 \cdot 288$ clock ticks and only from then is the output taken.

3.2.2 Salsa and ChaCha

We start by describing the underlying PRG. The PRG takes as input a 256-bit seed s and a 64-bit initalisation vector IV and has two main components a padding function `pad` and a permutation π. `pad` takes as input the seed, the IV and a 64-bit counter to form a 512-bit block. The permutation is based on three operations, addition (modulo 2^{32}, bitwise rotation and XOR. The PRG outputs bits in 512 bit blocks as needed, where the ith block r_i is computed as $h_i \leftarrow \texttt{pad}(s, i, IV)$ and $r_i \leftarrow \pi(h_i) \oplus h_i$.

3.3 Generating random bits in practice

We've seen how a PRG can stretch a random seed into a longer bit string that looks random. But this requires the underlying seed to be random. We will also often assume in cryptography that we have access to some source of randomness for generating keys or IVs. Random numbers are generated using a random number generator (RNG). A key difference between a PRG and a RNG is that entropy is continuously added to the internal state of the RNG. This source of entropy can come from a number of sources including keyboard events, mouse events or hardware interrupts.

Lecture 4

Block ciphers - intro

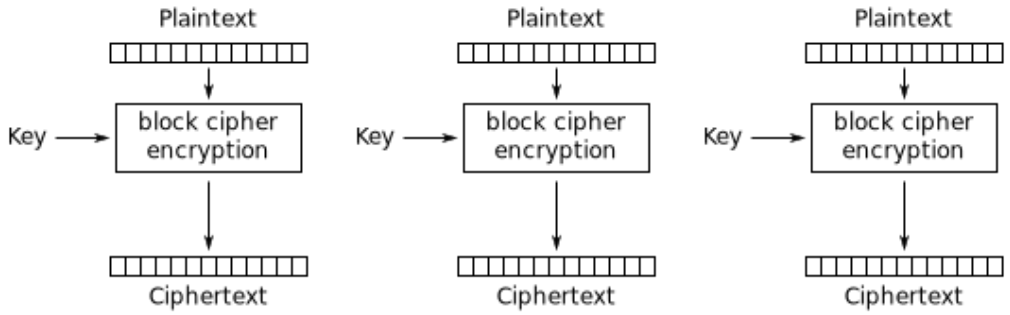
4.1 Definitions and properties

An n-bit block cipher is a pair of deterministic functions $\mathcal{E} = (E, D)$ $E : \{0,1\}^n \times \mathcal{K} \to \{0,1\}^n$, such that for each key $K \in \mathcal{K}$, $E(M, K)$ (the encryption function for K) is a bijection from $\{0,1\}^n$ to $\{0,1\}^n$ (denoted f_k). Thus f_k is a permutation on $\{0,1\}^n$, and decryption can be defined similarly using the inverse permutation f_k^{-1}.

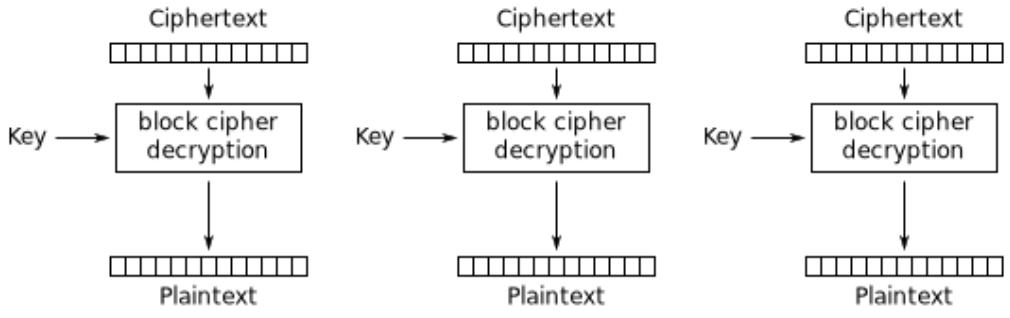
4.1.1 Modes of operations

A block cipher is not a cryptosystem on its own. It is only used to encrypt a particular block of data, and usually these blocks are much shorter than the messages we want to encrypt (because the block length can be at most the key length, and we want reasonably short keys). To form a full cryptosystem (a scheme for encoding and decoding text of any length), we must specify a mode of operation.

Electronic Codebook The simplest of the encryption modes of operation is the electronic codebook (ECB) mode (named after conventional physical codebooks). The message is divided into blocks, and each block is encrypted separately.

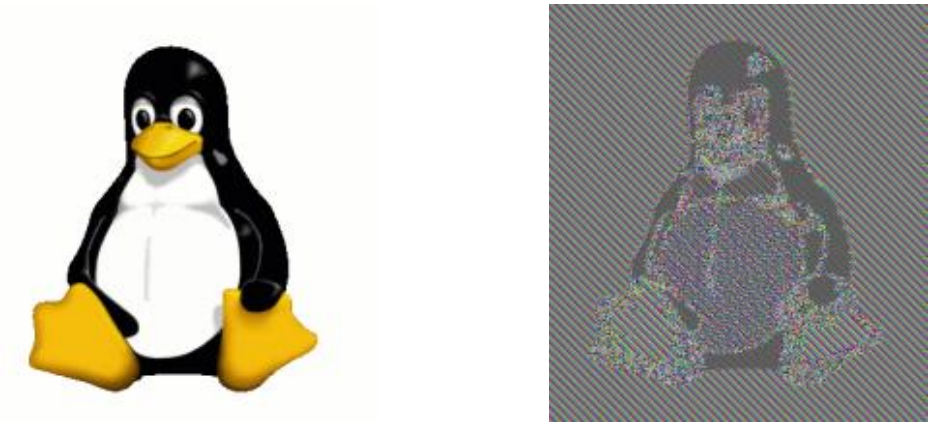


Electronic Codebook (ECB) mode encryption



Electronic Codebook (ECB) mode decryption

The disadvantage of this method is a lack of diffusion. Because ECB encrypts identical plaintext blocks into identical ciphertext blocks, it does not hide data patterns well Therefore, global patterns across the plaintext are still kept in the ciphertext. As such, ECB is not recommended for use in cryptographic protocols.



4.1.2 Feistel cipher

A Feistel cipher is an iterated cipher mapping a $2t$-bit plaintext (L_0, R_0), for t-bit blocks L_0 and R_0, to a ciphertext (R_r, L_r), through an r-round process where $r \geq 1$.

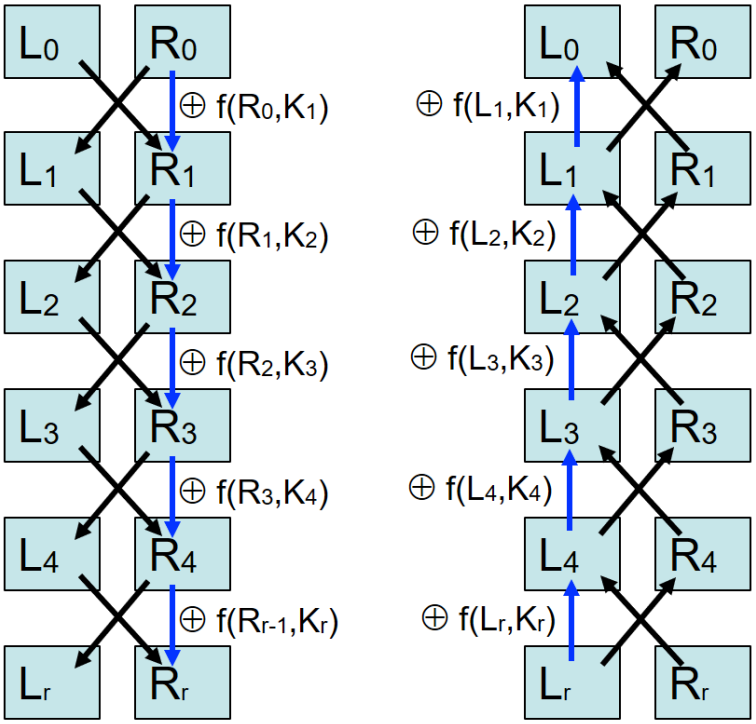
For $1 \leq i \leq r$, round i maps $(L_{i-1}, R_{i-1}), K_i \to (L_i, R_i)$ as follows:

$$L_i = R_{i-1},$$
$$R_i = L_{i-1} \oplus f(R_{i-1}, K_i),$$

where each subkey K_i is derived from the cipher key K.

Typically in a Feistel cipher, $r \geq 3$ and often is even. Decryption is achieved using the 'mirror image' of the same r-round process but with subkeys used in reverse order, K_r through K_1. Often, the Feistel structure is used with the ciphertext output ordered as (R_r, L_r) rather than (L_r, R_r); the blocks are exchanged from their usual order after the last round. In this case, decryption is exactly the same process as encryption (but again with subkeys in reverse order).

The f function of the Feistel cipher need not be invertible to allow inversion of the Feistel cipher - it can be any function (though some choices will result in the cipher being insecure). We illustrate that successive rounds of a Feistel cipher operate on alternating halves of the ciphertext, while the other remains constant.



Properties of a good cipher The following properties are necessary (though not sufficient) for a secure cipher:

- Confusion, identified by Claude Shannon in his 1945 classified report A Mathematical Theory of Cryptography, is the property that each bit of the ciphertext should be dependent on multiple bits of the key. This property is notably not satisfied by any of the historical schemes from Lecture 1, which is partly the source of their insecurity.
- Diffusion, also from the same report, is the property that changing a bit of the plaintext should affect many of the bits of the ciphertext. The Autokey Vigenère cipher had this property, but the earlier schemes did not.
- Nonlinearity is also important, since ciphers which are linear functions (or close to them) are vulnerable to linear and differential cryptanalysis (Lecture **??**). For functions on binary strings, taking XOR $\oplus$ and permuting the bits of a string are both linear operations (and so is any composition of them). So, secure ciphers cannot be composed only of $\oplus$ and permutations, and must also include some nonlinear elements.

4.2 Data Encryption Standard

4.2.1 DES

The Data Encryption Standard (DES) is a Feistel cipher which processes plaintext blocks of $n = 64$ bits, producing 64-bit ciphertext blocks.

DES key schedule The effective size of the secret key K is $k = 56$ bits. More precisely, the input key K is specified as a 64-bit key, 8 bits of which (bits $8, 16, \ldots, 64$) are used as odd-parity bits:

$$\begin{aligned} k_8 &= k_1 \oplus k_2 \oplus k_3 \oplus k_4 \oplus k_5 \oplus k_6 \oplus k_7 \oplus 1 \\ k_{16} &= k_9 \oplus k_{10} \oplus k_{11} \oplus k_{12} \oplus k_{13} \oplus k_{14} \oplus k_{15} \oplus 1, \\ &\vdots \\ k_{64} &= k_{57} \oplus k_{58} \oplus k_{59} \oplus k_{60} \oplus k_{61} \oplus k_{62} \oplus k_{63} \oplus 1. \end{aligned}$$

The 2^{56} keys implement (at most) 2^{56} of the $2^{64}!$ possible bijections on 64-bit blocks. A widely held belief is that the parity bits were introduced to reduce the effective key size from 64 to 56 bits, to intentionally reduce the cost of exhaustive key search by a factor of 256.

Algorithm DES key schedule
INPUT: 64-bit key $K = k_1 \ldots k_{64}$.
OUTPUT: sixteen 48-bit keys K_i, $1 \le i \le 16$.
1. Define v_i, $1 \le i \le 16$ as follows: $v_i = 1$ for $i \in \{1, 2, 9, 16\}$; $v_i = 2$ otherwise.
2. $(C_0, D_0) \leftarrow PC1(K)$.
3. For i from 1 to 16, let $X^{\leftarrow v}$ denote X shifted cyclically to the left v times, then do

$$\begin{aligned} C_i &\leftarrow C_{i-1}^{\leftarrow v_i}, \\ D_i &\leftarrow D_{i-1}^{\leftarrow v_i}, \\ K_i &\leftarrow PC2(C_i, D_i). \end{aligned}$$

$PC1$

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
above for C_i; below for D_i						
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

$PC2$

14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

This permutation of the key bits has two main functions: it allows the Feistel scheme to be used with a much shorter key than if all the subkeys were independent, and it also provides some confusion.

Feistel function for DES Encryption proceeds in 16 stages or rounds. From the input key K, sixteen 48-bit subkeys K_i are generated, one for each round. Within each round, 8 fixed, 6-to-4 bit substitution mappings (S-boxes) S_i, collectively denoted S, are used. The 64-bit plaintext is divided into 32-bit halves L_0 and R_0. Each round is functionally equivalent, taking 32-bit inputs L_{i-1} and R_{i-1} from the previous round and producing 32-bit outputs L_i and R_i for $1 \le i \le 16$, as follows:

$$\begin{aligned} L_i &= R_{i-1}, \\ R_i &= L_{i-1} \oplus f(R_{i-1}, K_i), \end{aligned}$$

where

$$f(R_{i-1}, K_i) = P(S(E(R_{i-1}) \oplus K_i)).$$

Here E is a fixed expansion permutation mapping R_{i-1} from 32 to 48 bits. P is another fixed permutation on 32 bits. Both of these permutations provide diffusion (and confusion) to the cipher. The tables below give E and P, that is

$$E(r_1, r_2, r_3, \ldots, r_{31}, r_{32}) = (r_{32}, r_1, r_2, \ldots, r_{32}, r_1)$$
$$P(t_1, t_2, t_3, \ldots, t_{31}, t_{32}) = (t_{16}, t_7, t_{20}, \ldots, t_4, t_{25}).$$

E

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

P

16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

The S_i S-boxes are given in the table below. The S-boxes are the only nonlinear components of the encryption, and as such are the crucial component of its security. In DES, the S-boxes were introduced as lookup tables, with no indication of why they were chosen like that in the first place.

They are applied as follows. First, let $B_i = (b_1, b_2, b_3, b_4, b_5, b_6)$, then row $= 2b_1 + b_6$, column $= 8b_2 + 4b_3 + 2b_4 + b_5$. Then look-up the table and write the entry in binary (on four bits). For instance, here is how to compute $S_1(011011)$:

$$B = 011011$$
$$\text{row} = 1$$
$$\text{column} = 13$$
$$S_1[1, 13] = 5 = 0101.$$

Here is how to compute the Feistel function f:

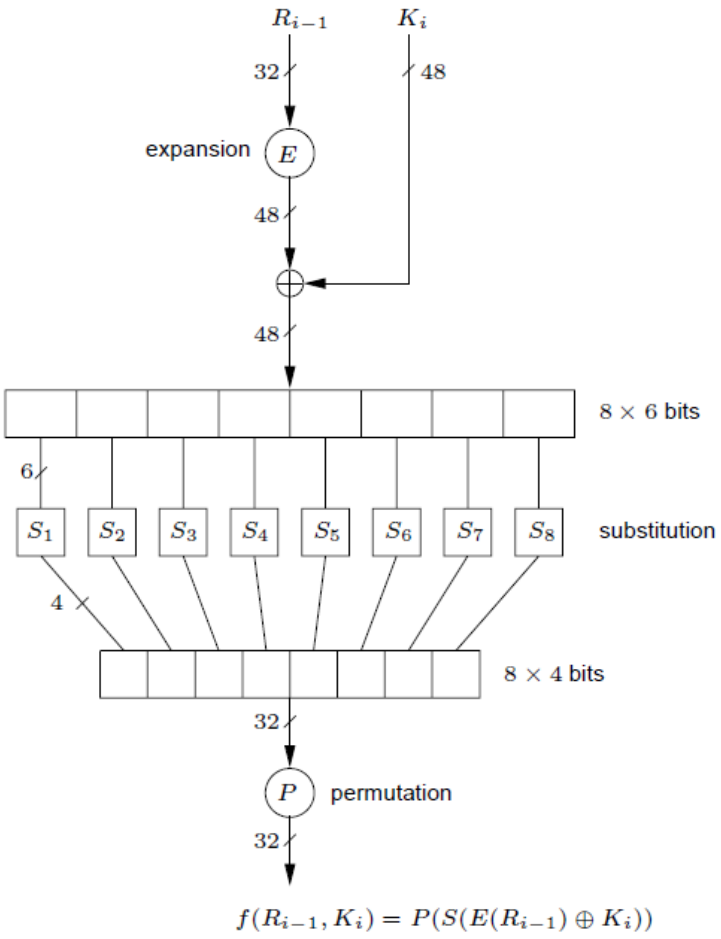
1. Expand $R_{i-1} = r_1 r_2 \ldots r_{32}$ from 32 to 48 bits using E: $T \leftarrow E(R_{i-1})$. (Thus $T = r_{32} r_1 r_2 \ldots r_{32} r_1$.)

2. $T \leftarrow T \oplus K_i$. Represent T as eight 6-bit character strings: $(B_1, \ldots, B_8) = T$.

3. $T \leftarrow (S_1(B_1), S_2(B_2), \ldots, S_8(B_8))$.

4. $T \leftarrow P(T)$.

row	column number															
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]	[15]
S_1																
[0]	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
[1]	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
[2]	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
[3]	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
S_2																
[0]	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
[1]	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
[2]	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
[3]	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9
S_3																
[0]	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
[1]	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
[2]	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
[3]	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12
S_4																
[0]	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
[1]	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
[2]	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
[3]	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14
S_5																
[0]	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
[1]	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
[2]	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
[3]	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3
S_6																
[0]	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
[1]	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
[2]	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
[3]	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13
S_7																
[0]	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
[1]	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
[2]	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
[3]	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12
S_8																
[0]	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
[1]	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
[2]	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
[3]	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11



The whole DES algorithm An initial bit permutation (IP) precedes the first round; following the last round, the left and right halves are exchanged and, finally, the resulting string is bit-permuted by the inverse of IP, sometimes referred to as final bit permutation $FP = IP^{-1}$. Similarly to the tables above, we have

$$IP(m_1, m_2, \ldots, m_{64}) = (m_{58}, m_{50}, \ldots, m_7)$$
$$FP(b_1, b_2, \ldots, b_{64}) = (b_{40}, b_8, \ldots, b_{25}).$$

IP

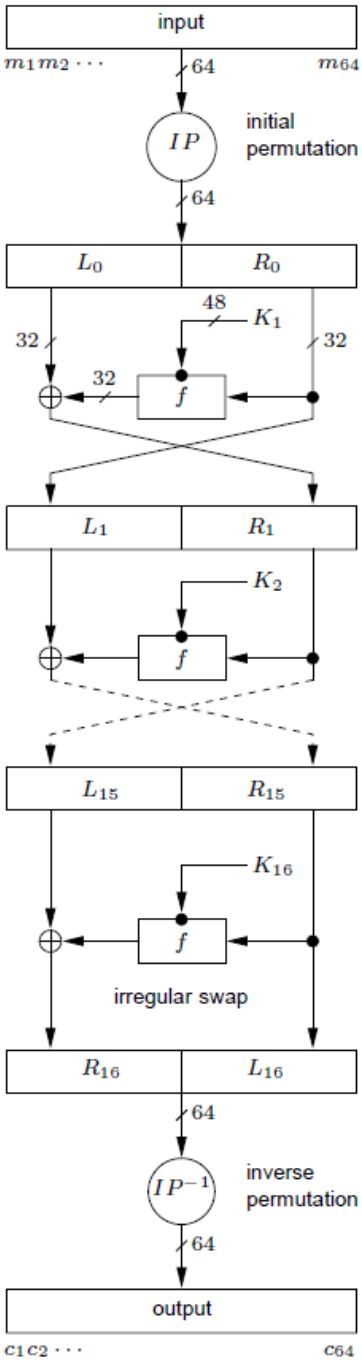
58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

FP

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

Algorithm Data Encryption Standard (DES)
INPUT: plaintext $M = (m_1, \ldots, m_{64})$; 64-bit key $K = (k_1, \ldots, k_{64})$.
OUTPUT: 64-bit ciphertext block $C = (c_1, \ldots, c_{64})$.
1. Compute sixteen 48-bit round keys K_i from K.
2. $(L_0, R_0) \leftarrow IP(M)$.
3. for i from 1 to 16, compute L_i and R_i as above.
4. $B = (b_1, b_2, \ldots, b_{64}) \leftarrow (R_{16}, L_{16})$.
5. $C \leftarrow FP(B)$.

Decryption involves the same key and algorithm, but with subkeys applied to the internal rounds in the reverse order.


input
$m_1 m_2 \cdots$
64
m_{64}
IP
initial permutation
64
L_0
R_0
48
K_1
32
32
32
f
L_1
R_1
K_2
f
L_{15}
R_{15}
K_{16}
f
irregular swap
R_{16}
L_{16}
64
IP^{-1}
inverse permutation
64
output
$c_1 c_2 \cdots$
c_{64}

Lecture 5

Attacking block ciphers

5.1 Weaknesses of DES

Attacking DES Even in 1977 the key size (56 bits) was considered 'low'. The NSA 'encouraged' the dropping of key size from 128 to 56 bits at the time - could they mount a brute force attack even then? Proposed machines to break DES by brute force are:

- 1977. Diffie and Hellman outline a $20million machine to crack DES in 1 day.
- 1993. Wiener proposed a $1million machine to crack DES in 7 hours.
- 1997. RSA corp. sponsored a competition to crack DES. It was won by a program using downtime of home PCs connected to the internet. 1 PC would have taken 2285 years to search the key space. They had 78,000 PCs signed up by the time they found the key, after searching 1/4 of the key space.
- 1998. The EFF built a $250,000 machine that cracks DES in 2 days.
- 2006. A $10,000 machine (COPACOBANA) is built from off-the-shelf kit that cracks DES in less than a week.

Key size If our key size is 56 bits, there are 2^{56} possible keys and testing randomly we would expect to test about 2^{55} before we cracked it. Suppose we can test a billion (2^{30}) possible keys per second. There are 2^{25} seconds per year, so we test about 2^{55} keys per year.

The amount of time required to break a 128-bit key is very much greater. About

$$2^{127} = 170,141,183,460,469,231,731,687,303,715,884,105,728$$

possibilities must be checked. A device that checks a billion billion keys (2^{60}) per second would require about 10^{13} years to exhaust the key space. This is a thousand times longer than the age of the universe.

AES permits the use of 256-bit keys. Breaking a symmetric 256-bit key by brute force requires 2^{128} times more computational power than a 128-bit key. A device that could check a billion billion (10^{18}) AES keys per second would require about 10^{51} years to exhaust the 256-bit key space.

5.2 Meet-in-the-middle attacks

Suppose we are using a k-bit cryptosystem with encryption function e and decryption function d. We decide to increase security by encrypting twice as follows.: choose two keys K_1 and K_2 each of k bits, and then do

$$C = e(e(M, K_1), K_2).$$

The effective security is not $2k$ bits. Suppose a plaintext/ciphertext pair M, C is publicly known. The attacker could compute a table of $e(M, K)$ for all 2^k possible keys K. Then she can compute $d(C, K')$ for all possible keys K', checking for a match:

This gives a pair of keys such that

$$e(e(M, K), K') = C.$$

The total encryptions/decryptions performed is 2^{k+1}. The space required is only enough to store 2^k keys and text snippets.

The meet in the middle attacks technique can be used in a more general fashion to attack many ciphers which consist of chained encryption operations. In particular, the two encryption functions used need not be the same. For example, if we encrypt using TripleDES:

$$C = e(d(e(M, K_1), K_2), K_3),$$

then we can enumerate $d(e(M, K_1), K_2)$ for all possible K_1 and K_2 (2^{2k} options, which is 2^{112} for DES), and then check for a match

$$d(e(M, K_1), K_2) = d(C, K_3),$$

which requires a further 2^{56} decryptions. Thus, the amount of encryptions/decryptions needed is only $2^{112} + 2^{56} << 2^{168}$.

5.3 Differential cryptanalysis

Differential cryptanalysis is a chosen-plaintext attack that involves comparing the XOR of two inputs to the XOR of the corresponding two outputs. The attacker is assumed to have a large number of tuples (x, x^*, y, y^*) (input, input, output, output), where the value $x' = x \oplus x^*$ is fixed and x and x^* are encrypted using the same key.

Differential cryptanalysis was discovered in the late 1980s by Biham and Shamir. They noted that DES was resistant to differential cryptanalysis. In fact, IBM and the NSA knew about differential cryptanalysis since the mid-70s and hence designed DES to be robust against this sort of attack.

Single S-box example As a toy example, consider the following S-box on 4 bits (Table 5.3):

Table 5.1: A small example of an S-box.

Input:	0000	0001	0010	0011	0100	0101	0110	0111
Output:	0000	1011	0101	0001	0110	1000	1101	0100
Input:	1000	1001	1010	1011	1100	1101	1110	1111
Output:	1111	0111	0010	1100	1001	0011	1110	1010

We look at all of the possible input differentials: XORs of two input strings. For a 4-bit S-box, there are 16 such differentials. For each of the input differentials, we count how often each of the 16 output differentials occurs. If our S-box was a truly random function, in expectation each would occur once. However, we will likely find that in practice, some occur significantly more often. In this example, for the input differential 1111 leads to the output differential 1010 *8 times* (e.g. when the pair of inputs is (0000, 1111) the output pair is (0000, 1010), which XOR to 1010. When the pair of inputs is (0100, 1011), the pair of outputs is (0110,1100), which also XOR to 1010, and so on).

Differential attacks on ciphers Now, consider attacking the following cipher structure, a simple substitution-permutation network:

Assume that the block length is 64 bits, and the three subkeys are independent and 64 bits each. A brute-force attack would need to guess one of $2^{64\times 3} = 2^{192}$ full keys. A meet in the middle attack would need 2^{128} guesses.

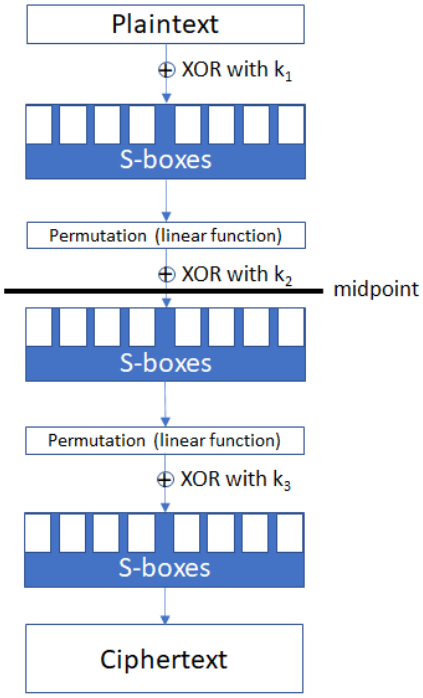


Figure 5.1: An example cipher structure.

Suppose now that we now analyse the differentials of the S-boxes. Since all of the other operations are linear, we can 'trace' differentials through the cipher structure to find input differentials that cause a particular differential to be significantly more common that expected later in the cipher. In particular, taking XOR with subkeys does not change differentials at all, and permutations change them only in a way we can entirely predict and invert.

More concretely, suppose we can find an input differential D_{in} that causes a particular differential D_{mid} to occur at the stage *after the XOR with* k_2 (we'll call this the midpoint of the cipher) with probability 2^{-50} (rather than the 2^{-64} we would expect from a random function; here the randomness is over choice of a random pair of plaintexts with input differential D_{in}).

We could use this to guess k_3 as follows: we take L pairs of random inputs with input differential D_{in}, and generate the corresponding ciphertexts (since this is a chosen plaintext attack, we assume we are able to do this). If we were able to guess k_3, we could decrypt up to the midpoint. And, with the correct guess of k_3, and using L sufficiently greater than 2^{50} (say, 2^{54}), we would find multiple pairs with differential D_{mid} at that point. On the other hand, with an incorrect guess of k_3, we (heuristically) expect the function to behave as random, which would mean we would get at most one pair with differential D_{mid}. So, now have a way of determining whether a guess for k_3 is correct.

So, we can now attack the cipher as follows: generate L pairs of plaintexts with differential D_{in}. For each possible guess of k_3, decrypt all of these pairs up to the midpoint and check how many differentials of d_{mid} we see - if it's significantly more than 1, then the guess must have been correct. Once we have the correct k_3, we find k_1 and k_2 using a standard meet-in-the-middle attack.

The complexity of this attack is dominated by the time taken to guess k_3. For this, we test 2^{64} guesses, and for each guess we decrypt 2^{54} pairs of ciphertexts, i.e. the amount of decryptions we need to make is $2^{64+54+1} = 2^{119}$, improving over the 2^{128} of a meet-in-the-middle attack.

5.4 Linear cryptanalysis

Linear cryptanalysis is a known-plaintext attack: it requires many plaintext-ciphertext pairs to be known by the attacker, but it does *not* require the attacker to be able to choose them. So, for example, if a single large file and its corresponding ciphertext are made public, this may prove sufficient for a linear attack. This makes linear cryptanalysis more practically realistic than differential cryptanalysis.

Recall that, in the context of operations on strings of bits, XOR ($\oplus$) is the only nontrivial bitwise linear operator. That means that if we were to have a linear encryption function $e : \mathcal{M} \times \mathcal{K} \to \mathcal{C}$, we would be able to express any bit of the ciphertext as the XOR of a selection of plaintext and key bits[1], for example:

$$c_{15} = p_1 \oplus p_3 \oplus p_{18} \oplus p_{22} \oplus k_2 \oplus k_7 \oplus k_{10} \ .$$

Linear functions do not make secure encryption functions, and we want our cryptosystems to be as far from linear as possible. In an ideal cipher, rather than having a subset of bits that always XOR to the same constant as above, every subset of bits should XOR to 0 half of the time and 1 the other half. That is, if we were to fix a uniformly random message and key, any linear equation concerning input and output bits would be true with probability 1/2. Our S-boxes (as the nonlinear components of the cipher) need to approximate this behaviour closely to be secure. If any linear equation concerning the input and output bits has too much bias (i.e. the probability that the equation is true is too far from 1/2), linear cryptanalysis can exploit this.

Linear cryptanalysis was introduced by Matsui in 1994. He actually performed a linear attack on DES the same year. The linear cryptanalysis of DES is a known-plaintext attack using 2^{43} plaintext-ciphertext pairs, all of which are encrypted using the same unknown key. This actual attack did not have any impact on the security of DES due to the large number of pairs required. Nonetheless, AES was designed to resist linear cryptanalysis.

Single S-box example Consider the S-box we looked at previously for differential cryptanalysis (Table 5.3). If it were secure, any linear equation concerning the 4 input bits (which we will denote x_1, x_2, x_3, x_4) and the four output bits (y_1, y_2, y_3, y_4) should be true with probability roughly 1/2 over a random input. However, we can find, for example, that the equation $y_1 = x_1 \oplus x_3 \oplus x_4$ is true for 14 of the 16 possible inputs, rather than the 8 that would be expected of a random function. This is a large bias, so this S-box is not very non-linear.

Linear attacks on ciphers Matsui found a linear relation in DES that would hold for 14 rounds, where (for some subset of key bits S_e,

$$m_R[17, 18, 24] \oplus c_L[7, 18, 24, 29] \oplus c_R[15] = k[S_e] \ .$$

This relation holds with probability $1/2 + 2^{-21}$.

We can extend this relation to the entire 16-rounds of DES by incorporating the first and last rounds, giving

$$(m_L \oplus F(k_1, m_R))\,[17, 18, 24] \oplus c_R[7, 18, 24, 29] \oplus (c_L \oplus F(k_16, c_R))\,[15] = k[S'_e] \ .$$

Bits 17,18 and 24 of $F(k_1, m_R)$ are part of just a single S-Box - therefore they only depend on six bits of k_1. $F(k_16, c_R)[15]$ depends on just one S-Box, and therefore also only depends on six bits of k_16. This means that $(m_L \oplus F(k_1, m_R))\,[17, 18, 24] \oplus c_R[7, 18, 24, 29] \oplus (c_L \oplus F(k_16, c_R))\,[15]$ depends on just 12 bits of the key k. If we were to test all possible combinations of these 12 bits on enough plaintext/ciphertext pairs, we'd see that for the correct key then the bias will be high and for the others it would be low.

[1] as well as, potentially, the constant 1, which has the effect of a logical NOT. In this case the function is really affine rather than linear, but we will not use that distinction in this course.

Lecture 6

AES

AES The Advanced Encryption Standard (AES) was introduced in 2002 in order to replace DES as the new standard for block ciphers. Unlike DES, where some details were kept secret, AES was selected using a competition where all the algorithms had to explain all the details and design choices.

AES is a substitution-permutation network, meaning that its structure consists of substitutions (S-boxes) and permutations (linear operations that shuffle the bits, providing confusion and diffusion), as will as adding subkeys. It is not a Feistel cipher, and so its S-boxes are required to be bijections so that the cipher is decryptable. Figure 6 is an example of a substitution-permutation network (but not AES's specifically, which has more rounds and a couple of extra steps).

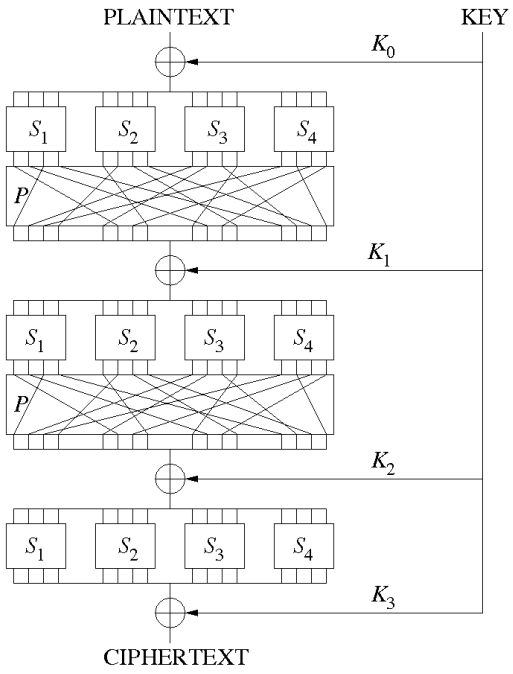


Figure 6.1: An example substitution-permutation network.

AES works on a 128 bit message block represented as a 4 by 4 array of bytes. The key length can be chosen to be either 128 bits, 192 bits, or 256 bits. AES applies a number of rounds (either 10, 12, or 14, depending on the key length) of:

- S-box transformation
- ShiftRows
- MixColumns
- Add sub-key.

Operations 1-3 are all fixed. The 'Add sub-key' steps are the only use of the key. Again, the S-box substitutions are the key non-linear element. The rest helps diffuse uncertainty.

Like DES, AES uses a key schedule to generate sub-keys for each round, since each requires 128-bit subkeys. The subkeys are generated from the overall key via a sequence of linear transformations and applications of the S-box.

Design of S-boxes Mathematically, an S-box is a function $\{0,1\}^n \to \{0,1\}^m$ for some m and n. It can then be viewed as a tuple of m Boolean functions $(s_1, \ldots, s_m)$, where $s_i : \{0,1\}^n \to \{0,1\}$ is a Boolean function.

The security of the block cipher depends on the S-box, and hence on its component Boolean functions. As such, different parameters of Boolean functions have been introduced: nonlinearity, balancedness, resiliency, correlation immunity, algebraic degree, etc.

Unlike DES, whose S-boxes are given only as lookup tables without justification, AES's S-boxes are derived algebraically. Specifically, they are based on the multiplicative inverse over $GF(2^8)$, known to have good non-linearity properties. AES is therefore more resistant to linear cryptanalysis (and is also resilient to differential cryptanalysis). So far, no practical (or even semi-practical) cryptanalysis attacks against AES are known (though side-channel attacks against some implementations are).

Lecture 7

Ideal primitives

7.1 Pseudo-random functions

Whilst we often use block ciphers as the main component in cryptographic systems, in most cases the right tool to use a pseudo-random function (PRF). Often we can use a block cipher in place of a PRF, under certain assumptions.

A pseudo-random function F is a deterministic algorithm defined over $(\mathcal{K}, \mathcal{X}, \mathcal{Y})$ that takes a key $k \in \mathcal{K}$ and an input block $x \in \mathcal{X}$, it outputs an output block $y \in \mathcal{Y}$, where $y = F(k, x)$. In the same way that we wanted the PRG (under a seed) to be a close as possible to something that would output randomly, we want the PRF (under a key) to be as close to possible as a random function. That is, choosing a random key k, the PRF should be close to choosing a function randomly from the set of all functions from $\mathcal{X}$ to $\mathcal{Y}$.

We define the PRF game following this notion - where an adversary has to decide if they're interacting with a PRF or with a random function.

PRF security

$k \xleftarrow{\$} \mathcal{K}$
$f_0 \leftarrow F(k, \cdot)$
$r_1 \xleftarrow{\$} \text{Func}[\mathcal{X}, \mathcal{Y}]$
$b \xleftarrow{\$} \{0, 1\}$
$\hat{b} \leftarrow \mathcal{A}^{f_b}()$
Return $b = \hat{b}$

Figure 7.1: The PRF Security attack game

Where $\text{Func}[\mathcal{X}, \mathcal{Y}]$ denotes the set of all functions with domain $\mathcal{X}$ and range $\mathcal{Y}$, and $\mathcal{A}^{f_b}$ represents an adversary who can make a bounded number of queries to the function f_b - we will say that $\mathcal{A}$ is q-query PRF adversary if it makes at most q queries to f_b.

PRF switching lemma. So when does a block cipher act as a PRF? We know it isn't a PRF (since a block cipher is a permutation). As long as the adversary only makes a small number of queries (i.e. $q < \sqrt{|\mathcal{Y}|}$) it turns out that we can use a **secure** block cipher as a PRF, where the distinguishing advantage is bounded by $\frac{q^2}{|\mathcal{Y}|}$.

7.2 Stream ciphers from block ciphers

PRF implies PRG. The existence of a secure PRF implies the existence of a secure PRG, i.e. if we can construct a secure PRF then we are able to construct a secure PRG. Let F be a PRF defined over $(\mathcal{K}, \mathcal{X}, \mathcal{Y})$, let $\ell \geq 1$ be a poly-bounded value, let $x_1, \ldots, x_\ell$ be the first ℓ elements of $\mathcal{X}$ in a lexicographic order. We can then construct a PRG G with seed space $\mathcal{K}$ and output space $\mathcal{Y}^\ell$, as

follows: for some $k \in \mathcal{K}$:

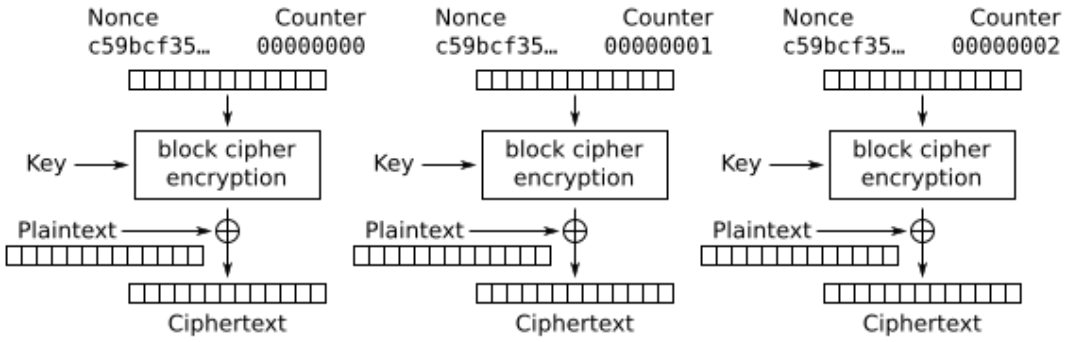
$$G(k) := (F(k, x_1), \ldots, F(k, x_\ell)).$$

If F is secure, then G is secure. Specifically, for every efficient adversary A against the PRG security game, there exists a ℓ-query adversary B against the PRF security game.

$\mathcal{B}$ makes queries $x_1, \ldots, x_\ell$ and receives $y_1, \ldots, y_\ell$ in response. $\mathcal{B}$ then runs $\mathcal{A}$, giving it the value $(x_1, \ldots, x_\ell)$. If $\mathcal{A}$ returns that the value is random, then $\mathcal{B}$ returns that F is a PRF, if $\mathcal{A}$ returns that the value is not random, then $\mathcal{B}$ returns that F is no a PRF.

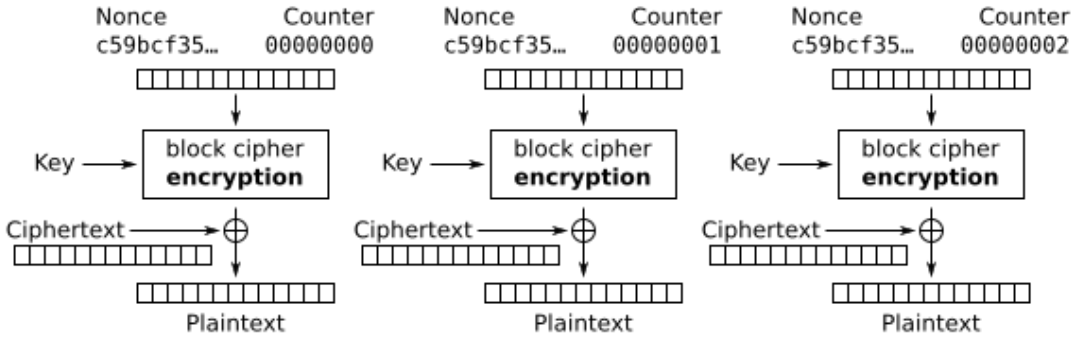
A new mode of operation. The above PRF construction gives us a method of building a secure stream cipher (with respect to the semantic security game) out of a block cipher! Let $\mathcal{E} = (E, D)$ be a secure block cipher defined over $(\mathcal{K}, \{0,1\}^n)$, then we can use the above construction to build a secure PRG, and from that build a secure stream cipher.

We can consider this as a new mode of operation for a block cipher (often referred to as counter mode). More formally, for counter mode we split the message being encrypted into ℓ n-bit blocks $m_0, \ldots, m_{\ell-1}$ (padding if needed). We then encrypt block i as $c_i = E(k, \langle i \rangle_n) \oplus m_i$. To decrypt the cipher block c_i, we compute $m_i = E(k, \langle i \rangle_n) \oplus c_i$.



Counter (CTR) mode encryption

[H]



Counter (CTR) mode decryption

It's clear that any block cipher operating in this mode is just acting as a stream cipher, using the block cipher as a PRG. Let $\mathcal{E}'$ be the stream cipher constructed from the block cipher $\mathcal{E}$ as above, then for any efficient adversary $\mathcal{A}$ in the semantic security game against $\mathcal{E}'$, there is an efficient adversary $\mathcal{B}$ in the BC game against $\mathcal{E}$ (where the BC game tries to distinguish between a block cipher and a permutation chosen at random - in a parallel to how the PRF game tries to distinguish between a PRF and a function chosen at random), such that

$$\mathbf{Adv}^{\mathrm{SS}}_{\mathcal{E}'}(\mathcal{A}) \leq 2\mathbf{Adv}^{\mathrm{BC}}_{\mathcal{E}}(\mathcal{B}) + \frac{\ell^2}{N} .$$

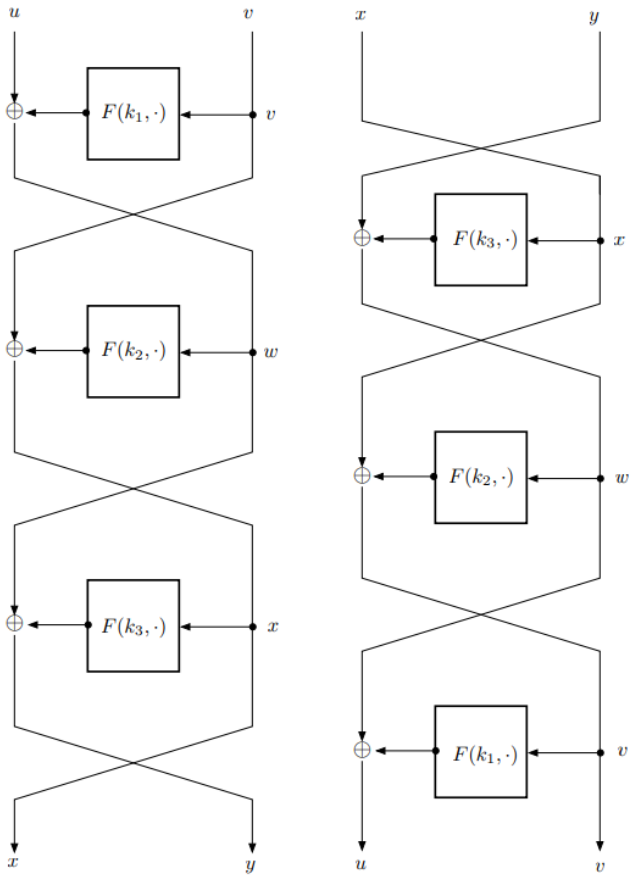
7.3 Block ciphers from PRFs and PRGs

We've shown that the existence of a secure block cipher implies the existence of a secure PRF, and that a secure PRF implies the existence of a secure PRG (and thus a secure stream cipher). We now show the reverse - that a secure PRG implies the existence of a secure PRF, which implies the existence of a secure block cipher! Whilst these results are mostly of theoretical interest (since we have practical constructions that we consider to be secure), these constructions highlight two main things - firstly a justification for the Feistel construction, and secondly hopefully helps to show the general interconnectedness of cryptographic primitives.

PRF implies Block Cipher. For any PRF defined over $(\mathcal{K}, \{0,1\}^n, \{0,1\}^n)$ we can construct a block cipher - this particular construction is known as the Luby-Rackoff construction. Let F be a secure PRF defined over $(\mathcal{K}, \{0,1\}^n, \{0,1\}^n)$ then we can build a secure block cipher defined over $(\mathcal{K}^3, \{0,1\}^{2n}, \{0,1\}^{2n})$. Let $(k_1, k_2, k_3) \in \mathcal{K}^3$ be the key and $(m_L, m_R) \in \{0,1\}^{2n}$ be the message, then we encrypt the message as follows:

$$\begin{aligned} w_1 &\leftarrow m_L \oplus F(k_1, m_R) \\ w_2 &\leftarrow m_R \oplus F(k_2, w_1) \\ w_3 &\leftarrow w_1 \oplus F(k_3, w_2) \\ (c_L, c_R) &\leftarrow (w_2, w_3) \ . \end{aligned}$$

Note that this construction is a Feistel cipher, where each round is replaced with the PRF F with a unique key.



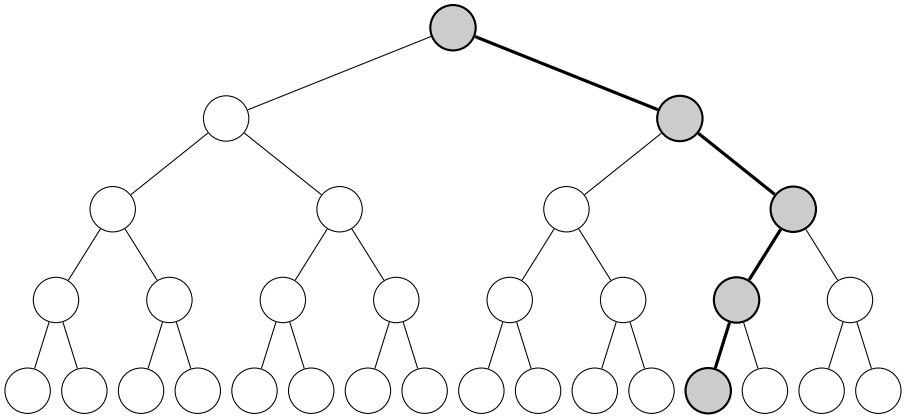
(a) Encryption (b) Decryption

Theorem 7.1. *If F is a secure PRF then the block cipher $\mathcal{E}$ build from F using the Luby-Rackoff construction is a secure block cipher.*

More formally, for every q-query adversary $\mathcal{A}$ against the BC security of $\mathcal{E}$, there exists a q-query adversary $\mathcal{B}$ against the PRF security of F, such that

$$\mathbf{Adv}^{\mathrm{BC}}_{\mathcal{E}}(\mathcal{A}) \leq \mathcal{O}(\mathbf{Adv}^{\mathrm{PRF}}_{F}(\mathcal{B}) + \frac{q^2}{2^n}) \ .$$

PRG implies PRF. We now look at building a PRF from a PRG. For simplicity, consider G to be a secure PRG that inputs an n-bit seed and outputs a $2n$-bit string. We will build a PRF F that takes an n-bit key and an n-bit message and outputs an n-bit ciphertext. To do this, we construct a complete binary tree of height n with 2^n leaves, where each node represents a call to the PRG. The "root" node PRG takes the key as is its seed, and outputs two blocks, each block is then the seed for the two children node PRGs - until we hit the final layer, where the two children (which are both leaf nodes) are just the two output message blocks. This tree will have 2^n leaf nodes - one leaf node for each corresponding message block. Encryption is done by traversing the tree to find the leaf node corresponding to the encrypted message, and the ciphertext is then the corresponding block generated by the PRG. Let $G_0(s)$ and $G_1(s)$ represent the first and second halves of the output from $G(s)$. Then the encryption of the message 0011 would be $G_0(G_0(G_1(G_1(k))))$.



7.4 Ideal Cipher Model

Often we want to be able to analyse the security of a larger construction or protocol that has a number of different primitives as building blocks. In these cases, we often care about one of the building blocks being a secure block cipher, rather than a specific block cipher. It might also be impossible to prove the security of these constructions for specific block ciphers. In these cases we instead prove that the construction is secure if the block cipher is an ideal cipher.

Idealism. Idealism refers to using the "ideal" version of a primitive. Typically this means one that behaves exactly as intended, i.e. an ideal PRF is one that is a truly random function. We'll see a few different idealised primitives later, when we talk about hashing.

Ideal Cipher Model. When analysing some larger scheme in the ideal cipher model, we effectively replace the block cipher $\mathcal{E} = (E, D)$ defined over $(\mathcal{K}, \mathcal{X})$ with a random keyed permutation Π. The game is modified as follow:

- At the start of the game, we sample the random keyed permutation.
- When the adversary would perform the encryption $E(k, x)$, they instead query $\Pi_k(x)$.
- When the adversary would perform the decryption $D(k, y)$, they instead query $\Pi_k^{-1}(y)$.
- The adversary can also make queries directly to Π and Π^{-1} for any pair (k, x) (or (k, y)).

Lecture 8

CPA

8.1 Extending the semantic security game

So far we've focused on how to encrypt a single message - either as a long stream or as a long message. We now want to extend this to allow us to encrypt multiple messages with the same key. In order to do this, we need to first extend our security games - note that semantic security only allows for one key to be used, and we know that semantically secure schemes can still be insecure if a key is used twice.

8.1.1 Multi key attacks

First lets consider the case where we want to encrypt multiple files or messages using a different key each time. Rather than the adversary choosing one pair of messages, and being asked to distinguish which one was encrypted. Instead we allow the adversary to select up to q pairs of messages, each pair is encrypted using a different, randomly chosen, key. The adversary is always given either m_0 or m_1, and is allowed to choose pairs adaptively - i.e. with knowledge of the response from other pairs.

Multi-key semantic security

$b \xleftarrow{\$} \{0,1\}$
$m_{10}, m_{11} \xleftarrow{\$} \mathcal{A}$
$k_1 \xleftarrow{\$} \mathcal{K}$
$c_i \xleftarrow{\$} E(k_i, m_{ib})$
for $i = 2$ to q :
  $m_{i0}, m_{i1} \xleftarrow{\$} \mathcal{A}(c_{i-1})$
  $k_i \xleftarrow{\$} \mathcal{K}$
  $c_i \xleftarrow{\$} E(k_i, m_{ib})$
$\hat{b} \xleftarrow{\$} \mathcal{A}(c_q)$
Return $b = \hat{b}$

Figure 8.1: Multi key security game

We can in fact show that any scheme that is secure with respect to the Semantic Security game is also secure with respect to the Multi-key Semantic Security game (providing q is at most polynomial).

Theorem 8.1. *If $\mathcal{E}$ is a semantically secure cipher then it is also a multi-key semantically secure cipher.*

More formally, for every q-query adversary $\mathcal{A}$ against the multi-key semantic security of $\mathcal{E}$, there exists an adversary $\mathcal{B}$ against the semantic security security of $\mathcal{E}$, such that

$$\mathbf{Adv}^{\mathrm{MSS}}_{\mathcal{E}}(\mathcal{A}) \leq q \cdot \mathbf{Adv}^{\mathrm{SS}}_{\mathcal{E}}(\mathcal{B}) \ .$$

The proof of this uses a hybrid argument similar to those we used to prove the security of repeating PRGs.

We'll now adapt the game slightly to make it clearer that this is the same as the adversary having oracle access to a "left-or-right" oracle. This oracle will, depending on b always encrypt either the left or the right message.

MSS	$\text{LR}_b(m_0, m_1)$
$b \xleftarrow{\$} \{0,1\}$	$k \xleftarrow{\$} \mathcal{K}$
$\hat{b} \xleftarrow{\$} \mathcal{A}^{\text{LR}_b}()$	$c_i \xleftarrow{\$} E(k, m_b)$
Return $b = \hat{b}$	Return c_i

Figure 8.2: Multi key security game - LR oracle

8.1.2 Chosen plaintext attack

We now consider the case where Alice doesn't want to encrypt every message with a different key, and instead wants to be able to send multiple messages. We'll now extend the initial semantic security game to one which allows for multiple encryptions - for ease of notation we'll use the "left-or-right" oracle approach.

SS-CPA (Multi-encryptions)	$\text{LR}_{b,k}(m_0, m_1)$
$b \xleftarrow{\$} \{0,1\}$	$c_i \xleftarrow{\$} E(k, m_b)$
$k \xleftarrow{\$} \mathcal{K}$	Return c_i
$\hat{b} \xleftarrow{\$} \mathcal{A}^{\text{LR}_{b,k}}()$	
Return $b = \hat{b}$	

Figure 8.3: Chosen Plaintext security game - LR oracle

Now the idea of having an oracle that only encrypts either the left or the right ciphertext might seem slightly contrived. However consider that if we query (m, m), the regardless of the value of b, the LR oracle will return the encryption of m!

8.1.3 IND-CPA.

It turns out that the multiple encryption game that we've described above is equivalent to the single encryption game. This allows us to consider this slightly simpler Semantic Security under Chosen Plaintext Attack game, this is often referred to as Indistinguishability under Chosen Plaintext Attacks, or IND-CPA for short.

SS-CPA
$b \xleftarrow{\$} \{0,1\}$
$m_0, m_1 \xleftarrow{\$} \mathcal{A}()$
$k \xleftarrow{\$} \mathcal{K}$
$c \xleftarrow{\$} E(k, m_b)$
$\hat{b} \xleftarrow{\$} \mathcal{A}^{E_k}(c)$
Return $b = \hat{b}$

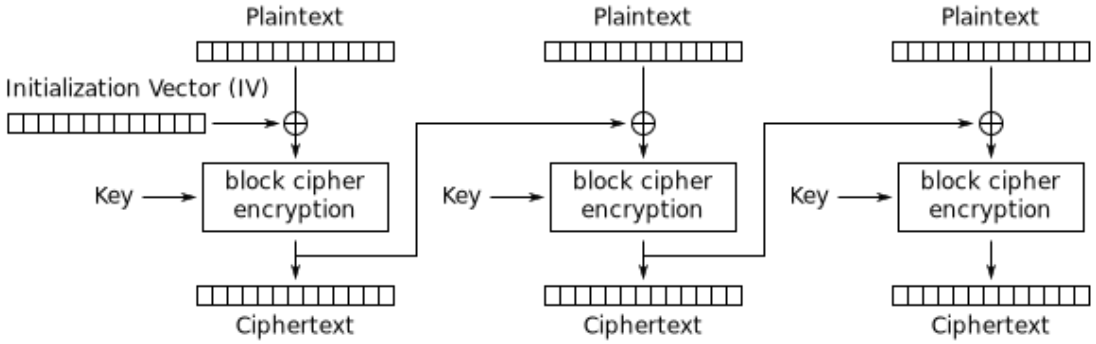
Figure 8.4: IND-CPA

8.2 Modes of operation

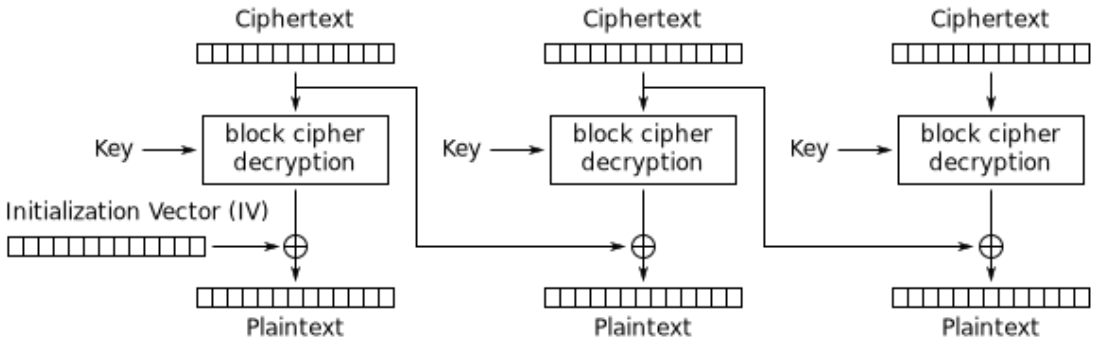
From the IND-CPA security game above, we can see that most block ciphers we've looked at would not be considered secure - primarily because they're deterministic. We can however make the block ciphers we've looked at IND-CPA secure by applying correct modes of operation.

8.2.1 Cipher Block Chaining

In the cipher block chaining (CBC) mode of operation, each block of plaintext is XORed with the previous ciphertext block before being encrypted. This way, each ciphertext block depends on all plaintext blocks processed up to that point. To make each message unique, a randomly chosen initialization vector (IV) must be used in the first block. This IV should be chosen differently every time (since repeatedly using a fixed IV is insecure), and is sent with the ciphertext since it is needed for decryption.



Cipher Block Chaining (CBC) mode encryption



Cipher Block Chaining (CBC) mode decryption

Whilst CBC mode isn't used as much currently, it is used in older version of TLS (e.g. TLS 1.0).

If the underlying block cipher is secure with respect to the BC game (i.e. can't be distinguish from a randomly chosen permutation) then that same block cipher in CBC mode is IND-CPA secure.

Theorem 8.2. *If $\mathcal{E} = (E, D)$ is a secure block cipher over $(\mathcal{K}, \mathcal{X})$, where $|\mathcal{X}| = N$ then the cipher $\mathcal{E}' = (E', D')$ over $(\mathcal{K}, \mathcal{X}^{\leq \ell})$ consisting of $\mathcal{E}$ in CBC mode is an IND-CPA secure cipher.*

In particular, for every q-query adversary $\mathcal{A}$ against the IND-CPA security of $\mathcal{E}'$ there exists an adversary $\mathcal{B}$ against the BC security of $\mathcal{E}$ such that

$$\mathbf{Adv}^{IND\text{-}CPA}_{\mathcal{E}'}(\mathcal{A}) \leq \frac{q^2\ell^2}{N} + \mathbf{Adv}^{BC}_{\mathcal{E}}(\mathcal{B}) \ .$$

Proof overview: We do a game hopping proof, starting with Game0 being the (multi encryption) Semantic Security CPA game. We then replace the Block Cipher being used with a random permutation. Finally we replace this with a PRF (using the PRF switching lemma) - where the 'queries' being made in total is $\mathcal{O}(q\ell)$.

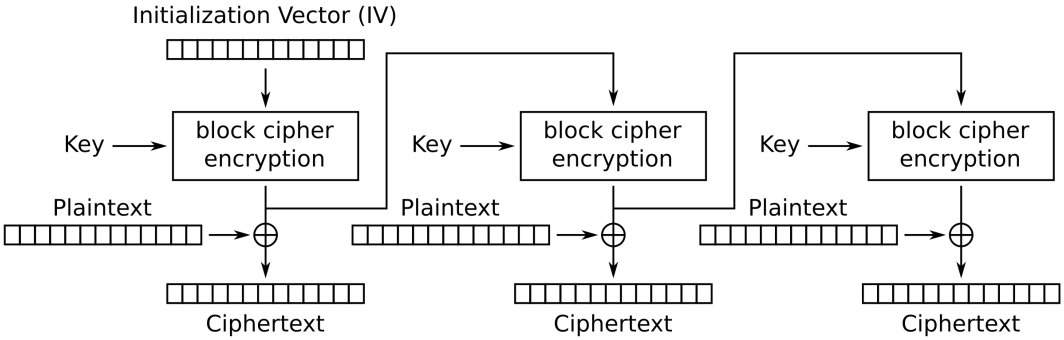
Padding So far we've assumed that every message will be a nice multiple of the block length - but what if it isn't? We need any padding function to be invertible so that when we decrypt we're able to remove the padding. TLS 1.0 defines the padding function (for AES, which has a 16 byte block size)

as follows: let the l be the size of the message in bytes, let $p = 16 - (v \bmod 16)$, we append p bytes to the message where each byte has the value $p - 1$. Note that this means that if the message is exactly the length of the block size, then we pad it with an entire block of bytes all set to the value 15.

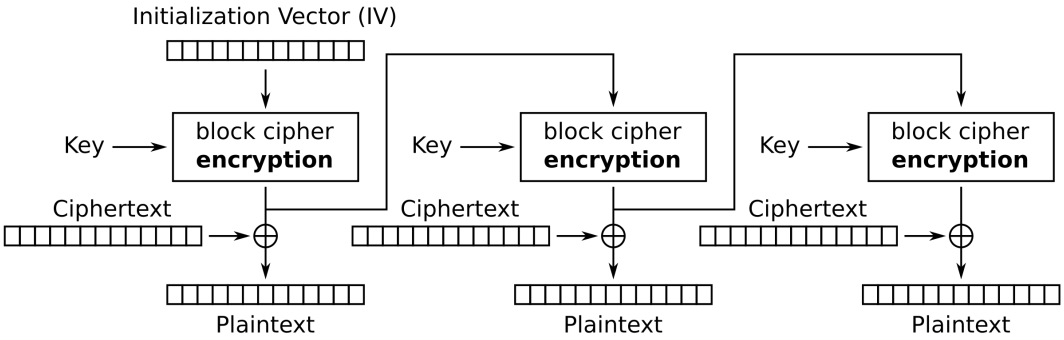
8.2.2 Further modes of operation

All of the following modes of operation also produce ciphers that are secure in the IND-CPA setting. In every case it requires the IV to be chosen randomly, i.e. the adversary submits a message and the IV chosen to encrypt it with, not the adversary submits a message and an IV.

Output Feedback The output feedback (OFB) mode, makes use of just the output of the block cipher and then encrypts this using the key. The ciphertext is then generated by XORing the plaintext with the output of each block cipher.

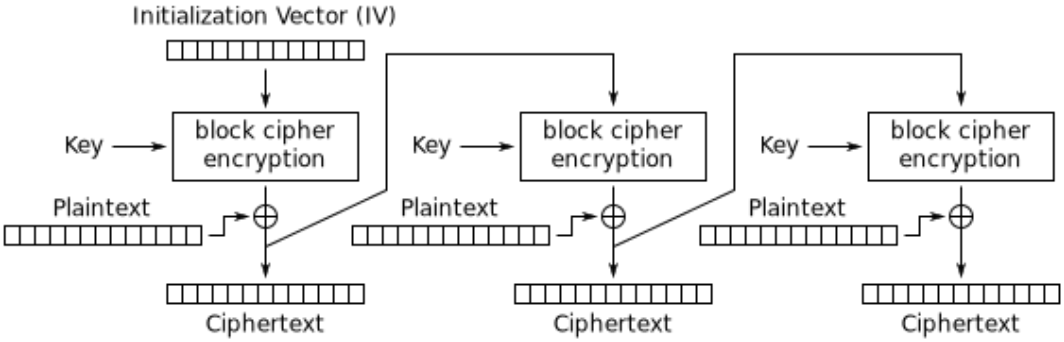


Output Feedback (OFB) mode encryption

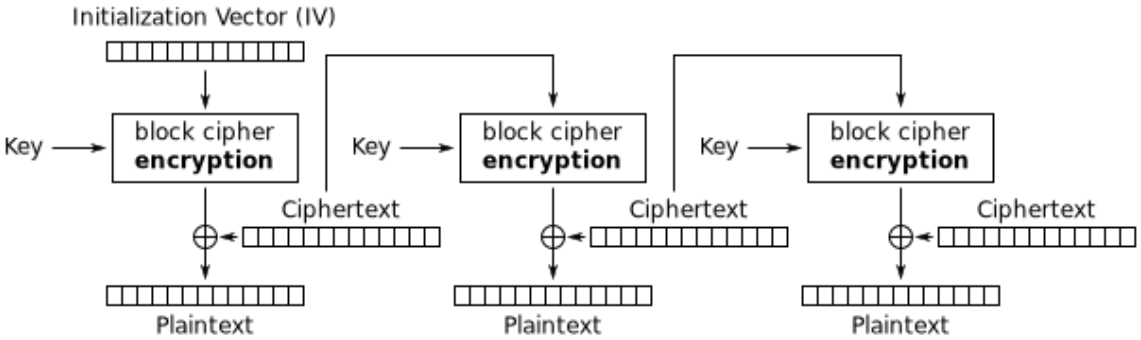


Output Feedback (OFB) mode decryption

Cipher Feedback The cipher feedback (CFB) mode, in its simplest variation is using the entire output of the block cipher. Note that this is subtly different from OFB, in that we don't just take the output of the block cipher, but the output XORed with the plaintext. In this variation, it is very similar to CBC.

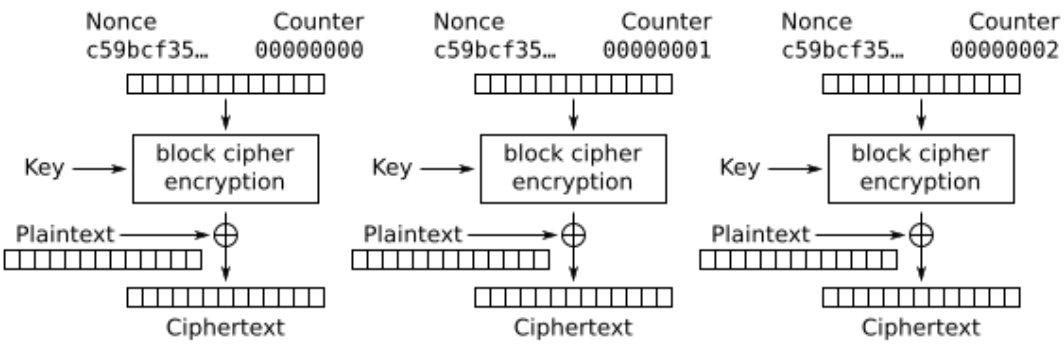


Cipher Feedback (CFB) mode encryption

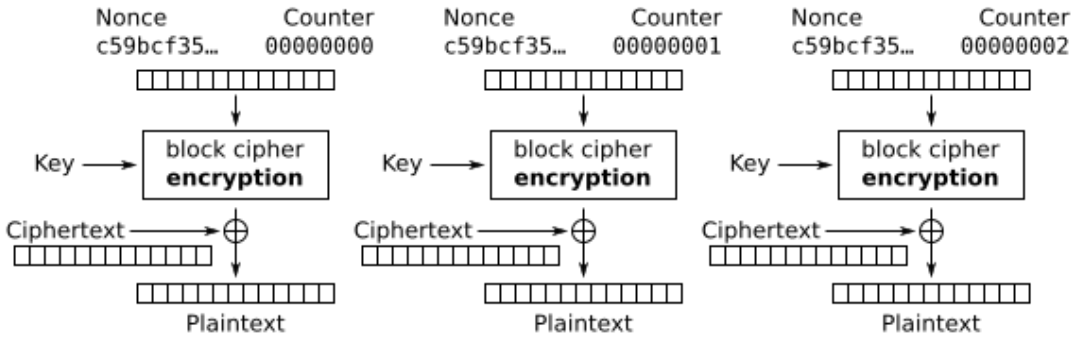


Cipher Feedback (CFB) mode decryption

Randomised counter mode This is almost identical to the counter mode we considered previously. The main difference is that a random IV is chosen, and this is then XORed with the counter value. This means that for every encrypted message, a different starting counter is used.



Counter (CTR) mode encryption



Counter (CTR) mode decryption

Lecture 9

Message Integrity

We've seen how parties can send encrypted messages that are secure against a passive adversary, but what about against an adversary that doesn't just eavesdrop, but modifies messages or adds new ones. When a user receives a message, we want the user to know if the message was authentic - i.e. is it from the person we think it is from, and has the message been modified.

9.1 MAC

In order to ensure the integrity of a message we use a message authentication code (MAC). The aim is to ensure that an adversary can't modify a message or inject a new one without the receiver detecting it. We assume that before communication happens, both users have managed to generate and share a secret key k. When a party want to send a message m, they compute a tag t based on m and k. The tag is computed using a MAC, i.e. the sender of the message m computes $t \leftarrow S_k(m)$ and sends (m, t). The receiver then verifies whether t is a valid tag on the message m with respect to k or not.

Definition More formally:

Definition 9.1 (Message Authentication Code)**.** *Formally a MAC consists of two probabilistic polynomial-time algorithms $\mathcal{I} = (S, V)$ such that:*

- *S takes as input a key k, a message $m \in \{0,1\}^*$, and a tag t, i.e. $t \leftarrow S_k(m)$.*
- *V is a deterministic algorithm that takes as input (k, m, t) and will accept a message if the tag is correct with respect to m and k.*

Typically, the verification algorithm recomputes the tag on m and k and checks if the new tag matches the original.

Security notions We now consider the corresponding security notion for MACs. An attacker might be able to influence the content of some messages, as such we allow the attacker to request the tag for any message. An attacker "breaks" the MAC scheme if it can compute a valid tag for a message that it did not request the tag for. A MAC that cannot be broken in the sense is said to be existentially unforgeable under a chosen-message attack. We formally define the forgeability game as follows.

Forge
$k \xleftarrow{\$} \mathcal{K}$
$(m, t) \xleftarrow{\$} \mathcal{A}^{\mathcal{O}}()$
return $V(m, t) \wedge m \notin Q$

$\mathcal{O}(m)$
$Q \leftarrow Q \cup m$
return $S_k(m)$

Figure 9.1: The S Forgeability Game

Replay attacks The above definition does not offer protection against replay attack, where an attacker simply resends a past message with its valid tag. For example if Alice sends a request to her bank to transfer £1,000 from her account to Bob, she'd compute the tag and send the message and tag. The bank can verify the tag is correct and send Bob the money. Whilst Bob can't edit the message to change the amount to £10,000 (as the tag would be invalid), Bob can send the original message and tag multiple times - which will still have the same effect.

A MAC by itself is unable to protect against replay attacks, as the verifier is stateless. Instead protection must be handled at some higher level. Two common techniques for preventing replay attacks are to use sequence numbers, or time stamps.

Timing attack on Verify It's important to ensure that if verify is using the canonical verification (i.e. recomputing the tag and comparing) that the final comparison check ($t = t'$) is done using a constant time approach. If it is not constant time, then the attacker can build up the tag byte by byte, using just 256 queries per byte of the tag. The MAC verification algorithm used by the Xbox 360 was susceptible to this timing attack (taking 2.2ms to compare each byte) and this allowed attackers to create valid tags for pirated games.

9.1.1 Constructing a MAC from a PRF - attempt 1

Let $F : \{0,1\}^n \to \{0,1\}^n$ be some pseudorandom function (in practice block ciphers are used). We construct a MAC for fixed length messages (length n), the function S computes $t \leftarrow F_k(m)$, and Verify uses the canonical verification algorithm.

This has the main disadvantage of only taking a fixed length message, in reality messages are likely to be much longer than the message size of the PRF being used. The natural idea is to split the message m into n bit blocks, and compute the MAC on each block individually - i.e. $t_i \leftarrow S_k(m_i)$, and the tag for m is the concatenation of each blocks tag - i.e. $t = t_1||t_2||\dots||t_{\lceil |m|/n \rceil}$.

Theorem 9.2. *Let F be a secure PRF defined over $(\mathcal{K}, \mathcal{X}, \mathcal{Y})$, then the deterministic MAC $\mathcal{I}$ produced from F using the above construction is a secure MAC.*

In particular for every q-query adversary $\mathcal{A}$ against the forgeability of $\mathcal{I}$, there exists a $(q+1)$-query adversary $\mathcal{B}$ against the PRF security of F such that

$$\mathbf{Adv}^{\mathrm{MAC}}_{\mathcal{I}}(\mathcal{A}) \leq \mathbf{Adv}^{\mathrm{PRF}}_{F}(\mathcal{B}) + \frac{1}{|\mathcal{Y}|} .$$

Proof. We prove this via game hopping, where Game 0 is the forgeability game with the specific S that we are using. For Game 1 we then replace F with a truly random function.

Game0	$\mathcal{O}(m)$
$k \xleftarrow{\$} \mathcal{K}$	$Q \leftarrow Q \cup m$
$f \leftarrow F(k, \cdot)$	return $f(m)$
$(m,t) \xleftarrow{\$} \mathcal{A}^{\mathcal{O}}()$	
return $V(m,t) \wedge m \notin Q$	

Game1	$\mathcal{O}(m)$
$k \xleftarrow{\$} \mathcal{K}$	$Q \leftarrow Q \cup m$
$f \xleftarrow{\$} \mathrm{Func}[\mathcal{X}, \mathcal{Y}]$	return $f(m)$
$(m,t) \xleftarrow{\$} \mathcal{A}^{\mathcal{O}}()$	
return $V(m,t) \wedge m \notin Q$	

The difference between the two games, i.e. $|\Pr[W_1] - \Pr[W_0]| = \mathbf{Adv}^{\mathrm{PRF}}_{F}(\mathcal{B})$. To see this more formally, we build the adversary $\mathcal{B}$ against the PRF game from $\mathcal{A}$ as follows. Adversary $\mathcal{B}$ responds to $\mathcal{A}$'s signing queries by forwarding them on to it's own PRF oracle. $\mathcal{A}$ will output a forgery (m,t),

where m was not one of the signing queries. $\mathcal{B}$ now forwards on m to the PRF oracle and receives some t'. If $t = t'$ then $\mathcal{B}$ returns that it's interacting with a PRF, if $t \neq t'$ then it believes it is interacting with a truly random function.

Finally we bound $\Pr[W_1]$. $\mathcal{A}$ is required to guess the value of f on some unseen input. Since f is truly random, the output on an unseen input is independent of the outputs on other points. Since the output is random, $\mathcal{A}$ will guess it with probability $\frac{1}{|\mathcal{Y}|}$.

Combing we get

$$\mathbf{Adv}^{\mathrm{MAC}}_{\mathcal{I}}(\mathcal{A}) = \Pr[W_0] \leq |\Pr[W_0] - \Pr[W_1]| + \Pr[W_1] \leq \mathbf{Adv}^{\mathrm{PRF}}_{F}(\mathcal{B}) + \frac{1}{|\mathcal{Y}|} .$$

□

This method however introduces a few more issues:

- An attacker could still reorder the blocks.
- An attacker could truncate the message, by dropping the last few blocks.
- An attacker could combine blocks from two different message/tag pairs.

Some potential solutions:

- We include the block index alongside each message block (prevents reordering).
- We include the number of blocks alongside each message block (prevents truncating).
- We include a random number with each message block (prevents interleaving).

Given a fixed-length MAC $\mathcal{I}' = (S', V')$ we combine these solutions to give the following MAC construction.

S:

- Given a key $k \in \{0,1\}^n$ and a message $m \in \{0,1\}^*$ of length $l < 2^{n/4}$.
- Choose a message id $r \in \{0,1\}^{n/4}$.
- Parse m as d blocks $m_1, \ldots, m_d$, each of length $n/4$ - padding the final block with 0s if needed.
- For each block compute $t_i \leftarrow S'_k(r||l||i||m_i)$, where i and l are encoded as strings of $n/4$ bits.
- Return $t = \langle r, t_1, \ldots, t_d \rangle$.

V:

- Given a key $k \in \{0,1\}^n$, a message $m \in \{0,1\}^*$ of length $l < 2^{n/4}$, and a tag $t = \langle r, t_1, \ldots t_{d'} \rangle$.
- Parse t and m as appropriate.
- Output 1 iff $d' = d$ and $V'_k(r||l||i||m_i, t_i) = 1$ for $1 \leq i \leq d$.

The intuition is that since S' is secure, an adversary can't introduce a new block with a valid tag. The extra information included in each block prevents the various attacks mentioned above. We prove security by showing that those attacks are the only ones possible.

Theorem 9.3. *Given a secure MAC $\mathcal{I}'$ (i.e.* $\Pr[\textsc{Mac-forge}_{\mathcal{I}'} = 1]$*), let $\mathcal{I}$ be the MAC produced by the above construction, then* $\Pr[\textsc{Mac-forge} = 1]$ *is negligible.*

Proof. Let REPEAT be the event that the same random identifier is used in two of the tags, let NEWBLOCK be the event that at least one of the blocks was never previously authenticated by S'.

$$\begin{aligned}\Pr[\text{MAC-FORGE} = 1] &= \Pr[\text{MAC-FORGE} = 1 \wedge \text{REPEAT}] \\ &\quad + \Pr[\text{MAC-FORGE} = 1 \wedge \overline{\text{REPEAT}} \wedge \text{NEWBLOCK}] \\ &\quad + \Pr[\text{MAC-FORGE} = 1 \wedge \overline{\text{REPEAT}} \wedge \overline{\text{NEWBLOCK}}] \\ &\leq \Pr[\text{REPEAT}] \\ &\quad + \Pr[\text{MAC-FORGE} = 1 \wedge \text{NEWBLOCK}] \\ &\quad + \Pr[\text{MAC-FORGE} = 1 \wedge \overline{\text{REPEAT}} \wedge \overline{\text{NEWBLOCK}}].\end{aligned}$$

We show that $\Pr[\text{REPEAT}]$ and $\Pr[\text{MAC-FORGE} = 1 \wedge \text{NEWBLOCK}]$ are negligible and that $\Pr[\text{MAC-FORGE} = 1 \wedge \overline{\text{REPEAT}} \wedge \overline{\text{NEWBLOCK}}] = 0$. This implies that $\Pr[\text{MAC-FORGE} = 1]$ is negligible.

As the value of r is chosen uniformly from a set of size $2^{n/4}$, the probability of REPEAT occurring can be calculated using a birthday bound argument. $\Pr[\text{REPEAT}] \leq q^2/2^{n/4}$, which is negligible.

If $\Pr[\text{MAC-FORGE} = 1]$ and REPEAT didn't occur, then NEWBLOCK must have occurred, thus $\Pr[\text{MAC-FORGE} = 1 \wedge \overline{\text{REPEAT}} \wedge \overline{\text{NEWBLOCK}}] = 0$.

As for $\Pr[\text{MAC-FORGE} = 1 \wedge \text{NEWBLOCK}]$, by definition if NEWBLOCK occurred, then a forgery of S' must have happened - and since S' is secure, the probability of this is negligable.

□

9.2 CBC-MAC - a more efficient approach

The constructions above show that it is possible to construct a secure message authentication code for arbitrary-length messages. This demonstrates, in principle, that secure MACs can be constructed from block ciphers. Unfortunately, the resulting construction is extremely inefficient: to compute a tag on a message of length dn, the block cipher is evaluated $4d$ times, and the tag is more than $4dn$ bits long. Fortunately, far more efficient constructions are available. We begin by exploring one such construction that relies solely on block ciphers.

CBC-MAC was one of the first message authentication codes to be standardized, we define a basic version of this.

Let $F : \{0,1\}^n \to \{0,1\}^n$ be some pseudorandom function (in practice block ciphers are used).

S:

- Given a key $k \in \{0,1\}^n$, a message $m \in \{0,1\}^*$ of length ln.
- Parse m into l blocks of length n.
- Set $t_0 \leftarrow 0^n$.
- For each m_i, compute $t_i \leftarrow F_k(t_{i-1} \oplus m_i)$.
- Return t_l.

V:

- If $|m| \neq ln$ then output 0.
- Return $t = S_k(m)$.

Whilst it is possible to extend the construction to allow for messages of different lengths, this construction is only secure when the lengths of the messages being authenticated is fixed and agreed upon in advance by the sender and receiver.

This construction is similar to the CBC-mode of encryption, with a few differences.

- CBC-mode uses a random IV, whilst CBC-MAC uses 0^n as the IV. Whilst a random IV is a requirement for the security of the CBC-mode of encryption, a random IV for CBC-MAC is not secure.

- In CBC-mode encryption all intermediate values t_i are output by the encryption algorithm as part of the ciphertext, whereas in CBC-MAC only the final block is output as the tag. If CBC-MAC is modified to output all the t_i obtained during the course of the computation then it is no longer secure.

9.2.1 Extending to arbitrary messages

There are two methods for extending the CBC-MAC construction above to allow for arbitrary-length messages.

1. Prepend the message m with its length $|m|$ encoded as an n-bit string. Then compute CBC-MAC on the result. Note that appending $|m|$ to the end of the message and then computing CBC-MAC is not secure.

2. Change the scheme to generate two independent keys, $k_1, k_2 \in \{0,1\}^n$. S computes CBC-MAC on m using k_1, with the output t, and then compute $t' \leftarrow F_{k_2}(t)$, returning t' as the tag for m.

9.3 Further reading

See Chapter 4.1-4.4 of [8] and Chapter 6 of [4]. Interestingly CBC-MAC is standardised both by NIST (FIPS PUB 113) and ISO (ISO/IEC 9797-1), it is worth reading both standards to see the differences in the way different organizations define the same thing!

Lecture 10

Hash functions

10.1 Definitions

Functions A function $f : \mathcal{D} \to \mathcal{R}$, maps each element of $\mathcal{D}$ to exactly one element of $\mathcal{R}$. Where $\mathcal{D}$ is the domain of f and $\mathcal{R}$ is the codomain of f. The range of f is the subset of elements in $\mathcal{R}$ that elements of $\mathcal{D}$ are mapped to.

A permutation $\pi : \mathcal{S} \to \mathcal{S}$, maps each element of $\mathcal{S}$ to exactly one, **unique** element of $\mathcal{S}$. The range, domain and codomain of π are all $\mathcal{S}$.

Hash Functions A *hash function* is a function that take inputs of some length and compress them into short, fixed-length outputs. The classic use of (noncryptographic) hash functions is in data structures, where they can be used to build hash tables that enable $\mathcal{O}(1)$ lookup time when storing a set of elements. A "good" hash function for this purpose is one that yields few collisions, where a collision is a pair of distinct elements x and x' for which $H(x) = H(x')$; in this case we also say that x and x' collide. (When a collision occurs, two elements end up being stored in the same cell, increasing the lookup time.)

We will consider two types of hash functions, keyed and unkeyed hash functions. We will only be considering hash functions whose domain is larger than their range.

Definition 10.1 (Keyed hash function)**.** *Formally a* keyed *hash function is a polynomial-time algorithm H, where H is a deterministic function that takes as input a key k and a string $x \in \{0,1\}^*$, and outputs a string $H_k(x) \in \{0,1\}^n$.*

Unlike the private key cryptography we've discussed so far, for hash functions the key is (generally) not secret, and we require the security notions to hold even when the adversary is given the key.

Definition 10.2 (Unkeyed hash function)**.** *An* unkeyed *hash function is a deterministic function $H : \{0,1\}^* \to \{0,1\}^n$.*

If the hash function is defined only for inputs $x \in \{0,1\}^{n'}$ $(n' > n)$ then we say that H is a fixed-length hash function, in this case we also call H a compression function.

10.2 Security notions

There are three basic notions of security for cryptographic hash functions: *preimage resistance*, *second-preimage resistance* and *collision resistance*.

We start with the weakest of these notions (i.e. the one that is easiest for a hash function to achieve) and move to the strongest notion.

Preimage resistance If a hash function is *preimage resistant*, also known as being *one-way*, then it should be hard to find a message with a given hash value. More formally, given a random key k and a random string y in the range of H it is computationally infeasible to find any value x in the domain of H_k such that

$$H_k(x) = y \ .$$

Second-preimage resistance A hash function is *second-preimage resistant* if given one message it should be hard to find another message with the same hash value. More formally given a random key k and a random string x in the domain of H it is computationally infeasible to find another value x' in the domain of H_k such that

$$H_k(x) = H_k(x') \ .$$

In particular it should take an adversary $\mathcal{O}(2^n)$ time to find a preimage/second-preimage.

Collision resistance A hash function is *collision resistant* if it should be hard to find two messages with the same hash value. Note that as we primarily consider hash functions where the domain is much larger than the range, collisions must *exist* but should be hard to find. More formally given a random key k it should be computationally infeasible to find two distinct values x and x' in the domain of H_k such that

$$H_k(x) = H_k(x') \ .$$

It is harder to construct collision resistant hash functions than one-way hash functions due to the birthday paradox. To find a collision of a hash function H, we can keep computing $H(x_1), H(x_2), H(x_3), \ldots$ until we get a collision. For a hash function that outputs strings of length n bits, then the probability of finding a collision after q queries to H is $q^2/2^{n+1}$, so we expect a collision after about $\sqrt{2^{n+1}}$ queries. The probability of finding a preimage of a hash function H, using a similar method as before, after q queries is $q/2^n$.

In order to be able to discuss the security of cryptographic hash functions, we write them as games as follows.

Preimage resistance	Second-preimage resistance	Collision resistance
$k \xleftarrow{\$} \textsc{Gen}(\lambda)$	$k \xleftarrow{\$} \textsc{Gen}(\lambda)$	$k \xleftarrow{\$} \textsc{Gen}(\lambda)$
$x \xleftarrow{\$} \{0,1\}^*$	$x \xleftarrow{\$} \{0,1\}^*$	$x, x' \xleftarrow{\$} \mathcal{A}^{H_k}()$
$y \leftarrow H_k(x)$	$x' \xleftarrow{\$} \mathcal{A}^{H_k}(x)$	return $H_k(x) = H_k(x')$ and $x \neq x'$
$x' \xleftarrow{\$} \mathcal{A}^{H_k}(y)$	return $H_k(x) = H_k(x')$ and $x \neq x'$	
return $H_k(x') = y$		

For the three above games we use $\mathbf{Adv}^{\textsc{Game}}_{H_k}(\mathcal{A})$ to denote the advantage that the adversary $\mathcal{A}$ has against $\textsc{Game}$, with access to the hash function H_k, compared to randomly guessing. i.e. $\mathbf{Adv}^{\textsc{Game}}_{H_k}(\mathcal{A}) = \Pr[\mathcal{A} \text{ wins } \textsc{Game}]$.

All security notions also apply to unkeyed hash functions. Cryptographic hash functions used in practice are generally unkeyed, and so from now on we now focus on unkeyed hash functions.

10.3 Separations and implications

We now give two lemmas to show that collision resistance implies second-preimage resistance, and that second-preimage resistance implies preimage resistance.

Lemma 10.3. *Assuming a function H is preimage resistant is a weaker assumption than assuming H is second-preimage resistant, i.e. for every adversary $\mathcal{A}$ that wins the preimage resistance game we can construct an adversary $\mathcal{B}$ that wins the second-preimage resistance game, specifically* $\mathbf{Adv}^{\textsc{preimage}}_{H}(\mathcal{A}) \leq \mathbf{Adv}^{\textsc{second-preimage}}_{H}(\mathcal{B})$.

Proof. $\mathcal{B}$ gets x as an input and hashes it to get y. $\mathcal{B}$ gives y to $\mathcal{A}$, which returns some x' such that $H(x') = y$. Since H has an (essentially) infinite domain, with high probability $x \neq x'$, and therefore we have won the SECOND-PREIMAGE game. □

Lemma 10.4. *Assuming a function is second-preimage resistant* ***for every value in the range*** *is a weaker assumption that assuming it is collision resistant i.e. for every adversary $\mathcal{A}$ that wins the second-preimage resistance game we can construct an adversary $\mathcal{B}$ that wins the collision resistance game, specifically* $\mathbf{Adv}^{\textsc{second-preimage}}_{H}(\mathcal{A}) \leq \mathbf{Adv}^{\textsc{collision}}_{H}(\mathcal{B})$.

Proof. Left as an exercise. □

It is possible to construct hash functions that are collision resistant but for **some** of the range of H it is not preimage resistant. As an example let G be a collision resistant hash function with outputs of length n. We define a new hash function H with output size $n+1$ bits as follows. For an input x, if $|x| = n$ then H returns $0||x$, else it returns $1||G(x)$. H is still collision resistant as G is collision resistant, however for any y that begins with a 0 we can invert, meaning it is not preimage resistant. This does **not** break the lemmas above, as the challenge value y is **not** chosen randomly from the range, rather we sample x randomly and **then** compute y. Considering the above construction, for an x sampled randomly from $\{0,1\}*$ the probability that the length of x equals n is negligible, and it is only values who's length is n that hash to something that can be inverted.

As collision resistance is the strongest notion, we typically focus only on hash functions that meet this security notion, although there are some applications where only preimage resistance is needed.

10.4 Further reading

See Boneh-Shoup [4] Chapters 7 and 8 for an overview that covers most of the next few weeks. For introductory information see Katz-Lindell [8] Chapter 6.1 or Smart [12] Section 14.1. For an overview of basic definitions and the relations between security notions, see [11].

Lecture 11

Hash Constructions I

11.1 Objectives

- Understand how the Merkle-Damgård transform works.
- Understand how the impact the padding scheme can have on Merkle-Damgård.
- Be able to prove the security/insecurity of slight variants of the Merkle-Damgård transform.
- Understand the Davies-Meyer construction.
- Have an understanding of how these two methods have been used to construct real world hash functions.

11.2 The Merkle-Damgård transform

For most applications we require cryptographic hash functions that can take arbitrarily long inputs. However it is a lot easier to construct fixed-length hash functions that only accept "short" inputs. The Merkle-Damgård transform allows us to convert a fixed-length hash function into a hash function that accepts arbitrarily long inputs. We refer to this technique as domain extension, and it is particularly interesting as it implies that compressing an input by a single bit is as easy as compressing by an arbitrary number of bits.

Let G be a fixed-length compression function for inputs of length $n + n' \geq 2n$ with output length n. Fix $l \leq n'$ and $IV \in \{0,1\}^n$. We construct a hash function H as follows. Given an initialisation vector IV and an input x of length $L \leq 2^l$:

1. Pad x using a (secure) padding scheme so that it is a multiple of n' bits.
2. Parse the resulting string as a sequence of b, n' bit blocks $x_1, \ldots, x_b$.
3. Set $z_0 = IV$.
4. For $i = 1$ to b, compute $z_i = G_k(z_{i-1}||x_i)$.
5. Return z_b

The string added to x to pad it is referred to as the *padding block* and is denoted PB. The values $x_1, \ldots, x_b$ are referred to as the message blocks, and the values $z_0, \ldots, z_b$ are called chaining variables.

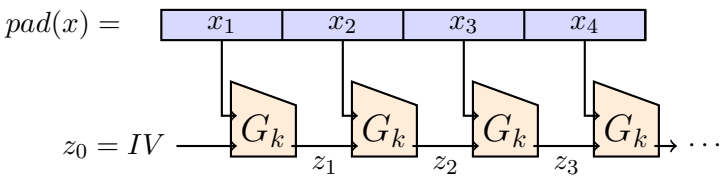


Figure 11.1: The Merkle-Damgård transform

Padding The standard Merkle-Damgård padding method is to add a padding block PB that is constructed as follows. PB consists of 1, enough 0's that $x||PB$ is l bits less than a multiple of n', and finally L encoded in binary, i.e. $PB = 100\ldots0||\langle L\rangle_b$. Note that if x starts as being a multiple of n', then a PB of length n' is added.

Theorem 11.1. *If G is a fixed-length, collision resistant, compression function, then H constructed as above is a collision resistant hash function i.e. for every adversary A that wins the collision resistance game for H there exists an adversary B that wins the collision resistance game for G, specifically* $\mathbf{Adv}^{CR}_H(A) \leq \mathbf{Adv}^{CR}_G(B)$.

Proof. We show that for any k, a collision in H_k yields a collision in G_k. Let x and x' be two different strings, of length L and L', respectively, such that $H_k(x) = H_k(x')$. Let $x_1, \ldots, x_b$ (respectively $x'_1, \ldots, x'_{b'}$) be the b (respectively b') blocks of the padded x (respectively x'). Let $z_0, \ldots, z_b, z'_0, \ldots, z'_{b'}$ be the corresponding chaining variables.

There are two cases to consider, where $L \neq L'$, and $L = L'$.

Case 1: $L \neq L'$. In this case, the final chaining variables are $z_b = G_k(z_{b-1}||x_b)$ and $z'_b = G_k(z'_{b'-1}||x'_{b'})$. Since $H_k(x) = H_k(x')$, we have $G_k(z_{b-1}||x_b) = G_k(z'_{b'-1}||x'_{b'})$. However $L \neq L'$, and so $x_b \neq x'_b$ - since the last l bits of each block encode L and L' respectively, thus $z_{b-1}||x_b$ and $z'_{b'-1}||x'_{b'}$ are a collision with respect to G_k.

Case 2: $L = L'$. This means that $b = b'$. Let j be the largest i for which $z_i \neq z'_i$. Since $|x| = |x'|$, but $x \neq x'$, there is an i such that $x_i \neq x'_i$ and so such a j certainly exists, and since $z_b = z'_{b'}$, $j < b$. Therefore $G_k(z_j||x_{j+1}) = G_k(z'_j||x'_{j+1})$ and since $z_j \neq z'_j$, we have a collision. □

Why not pad with 0's? Consider a Merkle-Damgård construction that pads the input string with 0s until it's the right length, with $n = n' = 4$. We give two distinct messages x and x', such that $H_k(x) = H_k(x')$ but there is no collision in G_k. Let $x = 10$ and $x' = 100$, in both cases they will be padded to 1000, giving $H_k(10) = H_k(100) = G_k(IV||1000)$.

As this shows, the padding method is an important part of the construction.

11.3 Building compression functions

Having shown how to make a collision resistant hash function that takes arbitrarily long inputs from a collision resistant, fixed length compression function, we now look at how to make a fixed length compression function from block ciphers.

Davies-Meyer compression functions Given a block cipher F with an n'-bit key length and an n-bit block length, we construct a compression function $G : \{0,1\}^{n'+n} \to \{0,1\}^n$ as follows:

$$G(x, m) = F_x(m) \oplus m \ .$$

Note that as the Davies-Meyer compression function is often used with Merkle-Damgård, it is common to see the function written as

$$m_i = G(x_i, m_{i-1}) = F_{x_i}(m_{i-1}) \oplus m_{i-1} \ .$$

Theorem 11.2. *If F is modeled as an ideal cipher, and G is constructed as above, then the advantage of any adversary A finding a collision for G is upper bounded as*

$$\mathbf{Adv}^{CR}_G(q) \leq O\left(\frac{q^2}{2^{n'}}\right)$$

This has some requirements on the block cipher, firstly that F_x should behave independently from $F_{x'}$, even if x and x' differ in only a single bit (i.e. no related key attacks). Secondly that F_m should behave randomly for all x, i.e. not possible to guess $F_x(m)$ without computing it. Neither of these properties were mentioned when discussing the security of block ciphers before, and indeed many block ciphers do not actually satisfy these properties.

11.4 Schemes

There are a number of schemes that have been designed using Merkle-Damgård, with the underlying compression function being created using the Davies-Meyer construction.

11.4.1 MD5

MD5 is a, now archaic, hash function that was designed by Ronald Rivest in 1991 using the Merkle-Damgård transform. The underlying compression function takes a 512-bit block and produces a 128-bit digest, the compression function consisted of 4 rounds, with 16 steps per round. Whilst originally believed to be collision resistant, collisions can now be found in under a minute on a standard PC. All the attacks make use of weaknesses in the underlying compression function. As such MD5 should not be used anywhere cryptographic security is needed, however sadly it is still being used today. A notable (known) real-world attack was the Flame malware, which was discovered in 2012 but which could have been active as early as 2010. For distribution the malware registered itself as a Microsoft update server, which was possible due to a forged Microsoft code signing certificate that still used MD5. Note that whilst collision resistance of MD5 is broken, there are no practical attacks on preimage resistance.

11.4.2 SHA-1

SHA-1 was published in 1995 and was designed by the NSA as a FIPS standard. It uses the Merkle-Damgård transform on a underlying compression function that was heavily inspired by MD5. The SHA-1 compression function takes a 512-bit block and produces a 160-bit digest. NIST deprecated SHA-1 in 2011 in favour of SHA-2 for applications where collision resistance is required, but still allows it in protocols such as HMAC whose security is based on other properties.

11.4.3 SHA-2

NIST updated its Secure Hash Standard in 2002 with SHA-2, a family of hash functions - SHA-256, SHA-384 and SHA-512 - where the suffix represents the digest size (in bits). The underlying compression function has a 256 bit state (for SHA-256) using 64 iterations or a 512 bit state (for SHA-384, SHA-512) using 80 iterations. Whilst a SHA-3 does exist, it coexists with SHA-2.

11.5 Further reading

See Katz-Lindell [8] sections 6.2 and 7.3.1 primarily, and Boneh-Shoup [4] sections 8.4 – 8.5 and Smart [12] section 14.3 for more information. For more information on the Merkle-Damgård transform, take a look at [10, **?**]

Lecture 12

Hash Constructions II and Attacks

12.1 Objectives

- Understand how the sponge construction works.
- Be able to choose the rate and capacity of the sponge based on desired security and speed requirements.
- Understand how preprocessing can be used to

12.2 Sponge

The Sponge construction is an alternative method of constructing a hash function, which uses a random permutation as its building block. Let π be the permutation $\pi : \{0,1\}^n \to \{0,1\}^n$, and let r and c be integers such that $r + c = n$. The integer r is the rate of the sponge, where a larger value of r leads to the hash function being evaluated faster. The integer c is the capacity of the sponge, where a larger value of c leads to stronger security.

We construct a hash function $H : \{0,1\}^L \to \{0,1\}^v$ as follows. Given an input m of length L:

1. Pad m so that it is a multiple of r bits - any injective padding scheme is sufficient.
2. Parse the resulting string as a sequence of b, r bit blocks $m_1, \ldots, m_b$.
3. $y_0 \leftarrow 0^n$
4. Absorption phase
   (a) For $i = 1$ to b,
   (b) $x_i \leftarrow y_{i-1} \oplus (m_i || 0^c)$
   (c) $y_i \leftarrow \pi(x_i)$
5. Squeezing phase
   (a) $h \leftarrow y_b$
   (b) $z_1 \leftarrow h[0 : r]$
   (c) for $i = 2$ to $\lceil v/r \rceil$
   (d) $h \leftarrow \pi(h)$
   (e) $z_i \leftarrow h[0 : r]$
6. Return $(z_1 || \cdots || z_{\lceil v/r \rceil})[0 : v]$

12.2.1 Random Permutation model

Theorem 12.1. *Let $H : \{0,1\}^L \to \{0,1\}^v$ be the hash function obtained from a permutation $\pi : \{0,1\}^n \to \{0,1\}^n$, with capacity c, rate r (such that $n = r + c$), and output length $v \leq r$, where 2^v and 2^c are super-polynomial, then the hash function H is collision resistant in the random permutation model.*

Specifically, for every collision finding adversary A, if the number of ideal-permutation queries, plus the number of $r-$bit blocks in the output messages of A is bounded by q, then $\mathbf{Adv}^{CR}_H(A) \leq \frac{q^2}{2^v} + \frac{q(q+1)}{2^c}$.

Proof of Sponge [Non-examinable]

Proof. Consider an attacker that has oracle access to a random permutation π and its inverse π^{-1}, and then outputs a pair of distinct messages; let $m = m_1, \ldots, m_b$ and $m' = m'_1, \ldots, m'_{b'}$ denote the result of the two messages after padding. [Note that due to the padding, these two messages are distinct.] We assume that the attacker never makes duplicate queries, and never queries $\pi^{-1}(y)$ once it knows $y = \pi(x)$ and visa versa, we further assume that the attacker makes all necessary queries to evaluate $H(m)$ and $H(m')$.

We define the following three events:

E1: The attacker makes two distinct queries to π whose results agree on their first v bits.

E2: The attacker makes a query to π or π^{-1} whose result has its last c bits equal to 0^c.

E3: The attacker makes two distinct queries to either π or π^{-1} whose results agree on the last c bits.

We start by justifying that if an attacker outputs a collision, then one of the three events occurred.

Consider the execution of the sponge construction on the padded message $m = m_1, \ldots, m_b$, let $y_0, \ldots, y_n, x_1, \ldots, x_n$ be the values of variables during execution as defined above. Define $y'_0, \ldots, y'_n, x'_1, \ldots, x'_n$ analogously with respect to the message $m' = m'_1, \ldots, m'_{b'}$.

If, for some i, the attacker queried $\pi^{-1}(y_i)$ to obtain x_i or $\pi^{-1}(y'_i)$ to obtain x'_i then we say an inverse query occurred.

Case 1: An inverse query occurred. Assume an inverse query occurred for the first message. Let i be minimal such that the attacker queried $\pi^{-1}(y_i) = x_i$. If $i = 1$ then the last c bits of x_1 are 0^c and **E2** occurred. Otherwise, the last c bits of $y_{i-1} = \pi(x_{i-1})$ and $x_i = \pi^{-1}(y_i)$ are equal and so **E3** occurred.

Case 2: No inverse query occurred. If $y_b \neq y'_{b'}$, then the first v bits of y_b and y'_b are equal, since a collision was outputted. Since no inverse queries occurred, the attacker must have queried $\pi(x_b)$ and $\pi(x'_{b'})$ and so **E1** occurred.

If $y_b = y'_{b'}$, we assume that $b' \geq b$. Let $C(z)$ denote the last c bits of the string z. Let i be maximal such that $(C(y_{b-i}), \ldots, C(y_b)) = (C(y'_{b'-i}), \ldots, C(y'_{b'}))$. If $i < b$, then $C(y_{b-i-1}) \neq C(y'_{b'-i-1})$ hence $x_{b-i} \neq x'_{b'-i}$, however

$$C(\pi(x_{b-i})) = C(y_{b-i}) = C(y'_{b'-i}) = C(\pi(x'_{b'-i}))$$

and so **E3** occurred. If $i = b$ and $b' > b$ then $C(\pi(x'_{b'-i})) = C(y'_{t'-i}) = C(y_0) = 0^c$, thus **E2** occurred. If $i = b$ and $b' = b$ then $(C(y_0), \ldots, C(y_b)) = (C(y'_0), \ldots, C(y'_b))$. Let j be minimal such that $m_j \neq m'_j$, then $y_{j-1} = y'_{j-1}$, but $x_j \neq x'_j$, and yet

$$C(\pi(x_j)) = C(y_j) = C(y'_j) = C(\pi(x'_j))$$

and so **E3** occurred.

We now bound the probabilities of each event occurring.

E1: this occurs when two distinct queries to π agree on the first v bits. Let $\mathrm{Coll}_{i,j}$ be the event that the results of the ith and jth queries of the attack agree on the first v bits. $\Pr[\mathrm{Coll}_{i,j}] \leq \frac{2^{n-v}}{2^n - 1} \leq 2 \cdot 2^{-v}$. There are $\binom{q}{2}$ possible pairs of queries, taking the union bound we have

$$\Pr[\mathbf{E1}] \leq \binom{q}{2} \cdot 2 \cdot 2^{-v} \leq \frac{q^2}{2^v} .$$

E2: this occurs when any query to π or π^{-1} has a result where the last c bits equal to 0^c. The probability of this occurring is $\frac{q}{2^c}$.

E3: this occurs when the results to two distinct queries to either π or π^{-1} agree on the last c bits. We apply a similar argument to that of **E1**, giving

$$\Pr[\mathbf{E3}] \leq \binom{q}{2} \cdot 2 \cdot 2^{-c} \leq \frac{q^2}{2^c} .$$

Applying the union bound we get $\mathbf{Adv}_H^{CR}(A) \leq \frac{q^2}{2^v} + \frac{q^2)}{2^c} + \frac{q)}{2^c}$. □

12.2.2 SHA-3/Keccak

Due to attacks on MD5 and SHA-1, NIST held workshops on the future of hash functions, and an open competition to design a new hash function was launched in 2007. Initially 64 submissions were entered (in 2008), of which 51 met minimum criteria. A second round started in 2009 with 14 submissions moving forward. In 2010 5 finalists were announced, and then in 2012 a winner - Keccak - was chosen. This was then published as SHA-3 in 2015.

Keccak uses a 1600 bit (unkeyed) permutation, with SHA3-256 having a rate r of 1088 and a capacity c of 512, SHA3-384 has $r = 832$ and $c = 768$, and SHA3-512 has $r = 576$ and 1024.

12.3 Attacks

12.3.1 Time/Space trade off

We consider an attack by an adversary who can perform some preprocessing - specifically preprocessing to try and invert a function, i.e. against preimage resistance.

We start by considering a fixed input hash function $H : \{0,1\}^l \to \{0,1\}^l$, given $y = H(x)$ an adversary has to find some x' such that $H(x') = y$. We typically care about how much computational resources were used up by the adversary **after** it has been given the challenge y to invert, we now consider one that may have performed a significant amount of preprocessing before y is known and then utilise the preprocessing to invert y.

We refer to the preprocessing phase as the 'offline' phase, and the phase after y is known as the 'online' phase.

We start by considering a trivial preprocessing algorithm that builds a table T. We iterate through values for x and compute $H(x)$. If $H(x)$ isn't in the table, then we add the pair $(H(x), x)$ to the table, sorted by $H(x)$. Then in the online phase, we find y in the table using a binary search and return the corresponding x'. This attack requires $\theta(2^l)$ space, but allows for inversion without any evaulations of H (except perhaps one to confirm that $H(x') = y$).

Contrast this to a brute force attack that would require $\theta(2^l)$ online time but no space. We will show that there is an approach that allows an attack to trade off time and space, specifically that by storing $O(2^{2l/3})$ values, an attacker can find a preimage in $O(2^{2l/3})$ time.

If H is a cycle We start by considering the case where H defines a cycle, i.e. for any starting point $x \in \{0,1\}^l$, that $x, H(x), H(H(x)), ...$ covers all of $\{0,1\}^l$. For clarity, let $N = 2^l$ and let $H^i(x)$ refer to the i-fold evaluation of H on x i.e. $H^3(x) = H(H(H(x)))$.

In the preprocessing phase, the attacker splits the cycle into $\sqrt{N}$ segments of length $\sqrt{N}$ each. The attacker can then build a table that stores that start and end of each segment, specifically the table consists of pairs of the form $(x_i, H^{\sqrt{N}}(x_i))$, sorted by the second value - this table contains $\sqrt{N}$ such pairs.

During the online phase the attacker is given a point y to invert, and checks $y, H(y), H^2(y)...$ against each endpoint stored in the table. Since y is in some segment, it must find an endpoint within $\sqrt{N}$ steps. Once an endpoint $H^{\sqrt{N}}(x_i)$ is identified, the attacked takes the corresponding starting

point x_i then computes $H(x_i), H^2(x_i), ...$ until y is reached, with the penultimate value giving the desired preimage, this takes at most $\sqrt{N}$ evaluations of H.

This attack will always find a preimage, and uses $O(\sqrt{N}) = O(2^{l/2})$ space and $O(\sqrt{N}) = O(2^{l/2})$ online hash computations.

Note that this works for any permutation, not just a permutation that defines just one cycle. The permutation can be decomposed into cycles, for each cycle that is of length less than $\sqrt{N}$, any point on the cycle would be inverted during the online phase (as it would require less than $\sqrt{N}$ computations to invert), for each cycle of length at least $\sqrt{N}$ the above technique can be used.

What if we had less space or time What if we were to limit the amount of space and (online) time that an attack has? This approach can be adapted, consider an attacker that can store $O(s)$ hashes, and can compute $O(t)$ hashes in the online phase. The attacker considers segments of length t and only chooses s such segments to store startpoint-endpoint pairs. The attacker is able to find any preimage that exists within those stored segments, as there are $s \cdot t$ values in those segments, and each value is unique, then this attack succeeds with probability $\frac{s \cdot t}{N}$.

But what about collisions? In most cases H will describe a cycle, it will, with high probability, have some number of collisions.

If we consider the above approach, then there is a chance that even if y is not in one of the segments, that as we keep computing H on y we collide with an endpoint. There is even a chance that two segments with unique starting points collide. Whilst these probabilities are small if the length of segments and the number of segments are small, if we were to try and create an attacker that was always successful at finding a preimage, then some improvements are required to the approach. The most common improvement made is to not build one table, but to build multiple tables, where for each table we generate the segments not by computing $x, H(x), H^2(x)...$, but by applying some secondary function f (where f is different for each table), between each evaluation of H, i.e. $x, f(H(x)), f(H(f(H(x)))), ...$ As the secondary functions are different for each table, segments can no longer fully collide. Whilst we will skip the full analysis of this, by creating $O(\sqrt[3]{N})$ tables, each with $O(\sqrt[3]{N})$ segments of length $O(\sqrt[3]{N})$, we can have an attacker that uses $O(N^{2/3})$ space and $O(N^{2/3})$ time to find a preimage to a random y with probability $\approx 1 - e^{-1} \approx 0.63$.

12.4 Further reading

See Katz-Lindell [8] section 7.3.3, and Boneh-Shoup [4] section 8.8 for the sponge construction, and Katz-Lindell [8] section 6.4.3 for Time Space tradeoffs. The original Hellman paper on Time Space tradeoffs for DES is an interesting read [7].

Lecture 13

Random oracles and Message Integrity II

13.1 The Random Oracle Model

As we've seen, there are multiple different ways of constructing hash functions. When we start applying hash functions as part of constructions, it would be useful if didn't have to prove that the construction is secure for each different hash function that exists. For this we use a heuristic called the **Random Oracle Model**.

In the random oracle model, we utilize a black box random oracle $\mathcal{O}$. The oracle takes a bit string x as input, and outputs a bit string y. Everyone, both honest parties and adversaries, has access to this oracle. The oracle is always consistent (i.e. if you query x to the oracle multiple times, then it will always return the same y.)

There are two equivalent ways of thinking about the oracle $\mathcal{O}$. The first is that $\mathcal{O}$ is chosen uniformly from the set of all functions on some specified domain and range. The second is that $\mathcal{O}$ is defined by a (initially empty) table, and when $\mathcal{O}$ queried on x, it first checks if some pair (x_i, y_i) is in the table. If it is in the table than y_i is returned, otherwise a new y is uniformly sampled and the pair (x, y) is added to the table.

13.1.1 The random oracle is collision resistant

We start by showing that a hash function modelled as a random oracle $\mathcal{O} : \{0,1\}^{n_{in}} \to \{0,1\}^{n_{out}}$, is collision resistant when $n_{in} > n_{out}$. i.e. for any adversary $\mathcal{A}$ that can make a polynomial amount of queries, its advantage in the following collision resistance game is negligible.

Collision resistance
$\mathcal{O} \xleftarrow{\$} Func(\{0,1\}^{n_{in}}, \{0,1\}^{n_{out}})$
$x, x' \xleftarrow{\$} \mathcal{A}^{\mathcal{O}}()$
return $\mathcal{O}(x) = \mathcal{O}(x')$ and $x \neq x'$

To see this we assume without loss of generality that $\mathcal{A}$ only outputs values x, x' that is has queried to the oracle, and that $\mathcal{A}$ never makes the same query to the oracle twice. Letting the queries of $\mathcal{A}$ be $x_1, \ldots, x_q$, where q is polynomial, it is clear that the probability that $\mathcal{O}(x_i) = \mathcal{O}(x_j)$ for some $j \neq i$. Since this is exactly equal to the probability that if we pick q strings $y_1, \ldots, y_q \in \{0,1\}^{n_{out}}$ independently and uniformly at random we have $y_i = y_j$ for some $i \neq j$. This is precisely the birthday problem, and so $\mathcal{A}$ succeeds with probability $O(q^2/2^{n_{out}})$.

13.2 MACs from Hash functions

We have already seen a construction of message authentication codes for arbitrary-length messages, namely CBC-MAC. We now consider another approach that relies on collision-resistant hash functions. We then discuss a standardized and widely used scheme called HMAC that can be viewed as a specific instantiation of this approach.

13.2.1 Hash-and-MAC

Collision-resistant hash functions can naturally be used for domain extension of message authentication codes. Say we have a fixed-length MAC for l-bit messages, and a collision-resistant hash function with l-bit output length. Then we can authenticate an arbitrary-length message m by using the MAC to authenticate the hash of m.

Given a MAC $\Pi = (\textsc{Mac}, \textsc{Vrfy})$ for messages of length l, and a hash function $H : \{0,1\}^* \to \{0,1\}^l$, we can construct a MAC $\Pi' = (\textsc{Mac}', \textsc{Vrfy}')$ for artbitrary-length messages as follows:
$\textsc{Mac}'$

- Given a key k and a message m
- Compute $t \leftarrow \textsc{Mac}_k(H(m))$

$\textsc{Vrfy}'$

- Given a key k, a message m, and a tag t.
- Return 1 iff $\textsc{Vrfy}_k(H(m), t) = 1$.

Intuitively, this is secure because the MAC ensures that the attacker cannot authenticate any new hash value, while collision resistance ensures that the attacker will be unable to find any new message that hashes to a previously used hash value.

Theorem 13.1. *Given a secure MAC Π for messages of length l, and a collision resistant hash function H, then the MAC Π' constructed as defined above is a secure MAC for arbitrary length messages.*

Informal. Let $\mathcal{Q}$ be the set of messages that have been authenticated using Π', and an attacker A is able to forge a valid tag for a new message $m^* \notin \mathcal{Q}$. Then either:

Case 1 : there is a message $m \in \mathcal{Q}$ such that $H(m) = H(m^*)$ (and thus A has found a collision in H).

Case 2 : for every $m \in \mathcal{Q}, H(m) \neq H(m^*)$, then A forged a valid tag on a "new message" $h^* = H(m^*)$ (and thus contradicting Π being a secure MAC).

□

13.2.2 HMAC

In principle, the hash-and-MAC approach from the previous section could be instantiated by combining an arbitrary collision-resistant hash function with the fixed-length MAC we discussed previously. This way of realizing the hash-and-MAC approach has at least two drawbacks in practice. First, it requires implementing two cryptographic primitives: a hash function and a block cipher. A more fundamental difficulty is that there is often a mismatch between the output length of hash functions and the block length of block ciphers.

We now try a couple of methods of building a MAC from just a hash function.

Prepend the key The first method is to prepend the key to the message and then hash, i.e. $\text{MAC}_k(m) := H(k||m)$. This works when we model H as a random oracle, however if H is built using Merkle-Damgård then this is insecure. Specifically, given $H(k||m)$ it is possible to calculate $H(k||M||PB||M')$ - where PB is the padding block for $k||m$.

Append the key The next consideration is to append the key to the message i.e. $\text{MAC}_k(m) := H(m||k)$. Whilst this is secure (assuming H is collision resistant), it allows for an offline attack when H is built using Merkle-Damgård. Specifically if an attacker can find m_0 and m_1 such that $H(IV||m_0) = H(IV||m_1)$, then this leads to a collision in $H(IV||m||k)$.

Prepend and append the key Both prepend and append the key, i.e. $\text{MAC}_k(m) := H(k||m||k)$. This solves the issues with both previous methods, and leads directly on to the design of HMAC.

HMAC construction The above concerns motivated the design of HMAC, a message authentication code for arbitrary-length messages that can be based on any hash function constructed using the Merkle–Damgård transform.

Given a secure hash function H constructed using Merkle–Damgård from a compression function $h : \{0,1\}^{n+n'} \to \{0,1\}^n$, where $n + n' > 2n + \log n + 2$, HMAC is constructed as follows. Fix distinct constants $\mathbf{opad}, \mathbf{ipad} \in \{0,1\}^{n'}$.

Mac

- Given a key k and a message $m \in \{0,1\}^*$
- Compute $t \leftarrow H\left((k \oplus \mathbf{opad})||H((k \oplus \mathbf{ipad})||m)\right)$.

Vrfy

- Given a key k, a message m, and a tag t.
- Return 1 iff $t = H\left((k \oplus \mathbf{opad})||H((k \oplus \mathbf{ipad})||m)\right)$.

The computation of HMAC on a message $m = m_1, m_2, \ldots$ using key k can be separated into an "inner" hash evaluation and an "outer" hash evaluation. The inner hash evaluation involves computing $\hat{m} \leftarrow H((k \oplus \mathbf{ipad})||m)$. The outer hash evaluation involves computation of the tag $t \leftarrow H\left((k \oplus \mathbf{opad})\hat{m}\right)$.

HMAC is the most widely deployed MAC on the Internet. It is used in SSL, TLS, IPsec, SSH, and a host of other security protocols. TLS and IPsec also use HMAC as a means for deriving session keys during session setup.

13.3 Further reading

For more information on KDFs see Boneh-Shoup [4] section 8.10, for more information on the Random Oracle methodology, see Katz-Lindell [8] section 6.5. The Random Oracle model was first introduced in [2] and gives a good explanation as to the need for this theoretical tool. For more information on the Game Playing technique see [3]

Lecture 14

Authenticated Encryption

Until now, we have considered how to obtain secrecy (using encryption) and integrity (using message authentication codes) separately. The aim of authenticated encryption, defined below, is to achieve both goals simultaneously. It is best practice to always ensure secrecy and integrity by default in the private-key setting. Indeed, in many applications where secrecy is required it turns out that integrity is essential also. Moreover, a lack of integrity can sometimes lead to a breach of secrecy.

Consider an authenticated (private-key) encryption scheme $\Pi = (\textsc{Enc}, \textsc{Dec})$. We define the encryption forgeability game as follows.

Forge	$\mathcal{O}(m)$
$k \xleftarrow{\$} \mathcal{K}$	$Q \leftarrow Q \cup m$
$(c) \xleftarrow{\$} \mathcal{A}^{\mathcal{O}}()$	return $\textsc{Enc}_k(m)$
$m \leftarrow \textsc{Dec}(c)$ return $m \neq \perp$	
$\wedge m \notin Q$	

Figure 14.1: The Encryption Forgeability Game

We say that a private-key encryption scheme is an authenticated encryption (AE) scheme if it is CCA-secure and unforgeable.

Often, a message m requires both secrecy and integrity but various associated data (e.g., header information) sent along with the message requires integrity only. While it is possible to simply concatenate the message and the associated data (in some way that allows for unambiguous parsing) and then use an AE scheme to encrypt them both, better efficiency can be achieved by providing the associated data with integrity protection only.

14.1 Generic constructions

It is tempting to think that any reasonable combination of a CPA-secure encryption scheme and a secure message authentication code should result in an authenticated encryption scheme. In this section we show that this is not the case. This demonstrates that even secure cryptographic tools can be combined in such a way that the result is insecure, and highlights once again the importance of definitions and proofs of security.

Let $\Pi_E = (\textsc{Enc}, \textsc{Dec})$ be a CPA-secure encryption scheme and let $\Pi_M = (\textsc{Mac}, \textsc{Vrfy})$ denote a secure MAC.

There are three natural approaches to combining the encryption scheme and the MAC.

1. Encrypt-and-authenticate
2. Authenticate-then-encrypt
3. Encrypt-then-authenticate

Encrypt-and-authenticate In this approach, encryption and message authentication are computed independently in parallel. Given a message m, the sender computes $c \leftarrow \text{ENC}_{k_E}(m)$ and $t \leftarrow \text{MAC}_{k_M}(m)$, and then returns $\langle c, t \rangle$. The receiver computes $m \leftarrow \text{DEC}_{k_E}(c)$, and (assuming $m \neq \perp$) computes $b \leftarrow \text{VRFY}_{k_M}(m, t)$. If $b = 1$ then the receiver outputs m, otherwise returns an error.

Authenticate-then-encrypt Here a tag t is first computed, and then the message and tag are encrypted together. That is, given a message m, the sender computes the tag, $t \leftarrow \text{MAC}_{k_M}(m)$ and then encrypts $c \leftarrow \text{ENC}_{k_E}(m||t)$. The receiver computes $m||t \leftarrow \text{DEC}_{k_E}(c)$, and (assuming $m||t \neq \perp$) computes $b \leftarrow \text{VRFY}_{k_M}(m, t)$. If $b = 1$ then the receiver outputs m, otherwise returns an error.

Encrypt-then-authenticate In this case, the message m is first encrypted and then a tag is computed over the result. That is, given a message m, the sender computes $c \leftarrow \text{ENC}_{k_E}(m)$ and $t \leftarrow \text{MAC}_{k_M}(m)$, and then returns the pair $\langle c, t \rangle$. The receiver computes $b \leftarrow \text{VRFY}_{k_M}(c, t)$ and if $b = 1$ then decrypts, i.e. computes $m \leftarrow \text{DEC}_{k_E}(c)$ and (if $m \neq \perp$) returns m.

14.1.1 Analysis of generic schemes

We analyze each of the above approaches when they are instantiated with "generic" secure components, i.e., when instantiated with an arbitrary CPA secure encryption scheme and an arbitrary strongly secure MAC. We stress that if an approach is rejected this does not mean that it is insecure for all possible instantiations of the components; it does, however, mean that any instantiation of the approach must be carefully analyzed and proven secure before it is used.

Encrypt-and-authenticate This method is problematic since it may not achieve even the most basic level of secrecy. Consider the tag t, the security notion (forgeability) for MACs does not make any guarantee about secrecy. For example, there could be a MAC that is secure under the forgeability notion but fully leak m as part of t. In a less contrived sense, for an encryption scheme to be CPA-secure it should not be possible for an eavesdropper to identify when the same message is sent twice. If the MAC is deterministic, then any time a message m is repeated, its tag will also be repeated.

Authenticate-then-encrypt The problem with this method surrounds how messages with a length that isn't exactly a multiple of n are handled. To give a concrete example of how this could be an issue, consider an encryption scheme used in CBC mode (which is CPA secure) and a secure MAC. This particular combination is badly broken - and lead a devastating attack on SSL 3.0 called POODLE.

Let us assume that underlying block cipher operates on 16 byte blocks. When encrypting a message using CBC mode, the message is padded, so that the length is a multiple of 128. One method (the one used in SSL 3.0) is if $p > 0$ bytes need to be padded, to pad the message with $p - 1$ arbitrary bytes and adds one additional byte whose value is set to $(p - 1)$, If the message length is already a multiple of the block length (16 bytes) then SSL 3.0 adds a dummy block of 16 bytes where the last byte is set to 15 and the first 15 bytes are arbitrary. During decryption the pad is removed by reading the last byte and removing that many more bytes.

Under the authenticate-then-encrypt method, we first compute the tag t for the message, and then encrypt $m||t$. In the case where this needs to be padded, the actual data being encrypted is $m||t||p$, where p is the padding. Note that the tag only protects the integrity of the message, and not the pad!

Suppose the adversary intercepts the (valid) ciphertext c for some unknown m. Assume that the length of $m||t$ is a multiple of 16 bytes. This means that a full padding block of 16 bytes is appended, where the final byte is 15.

The ciphertext will look as follows: $c = c_0, c_1, \ldots, c_{l_1}, c_l$, where c_0 is the IV, $c_1, \ldots, c_{l-1}$ is the encryption of $m||t$ and c_l is the encryption of the pad.

The attacker prepares a chosen ciphertext query $\hat{c}$ by replacing c_l with c_1. By definition of CBC decryption, decrypting the last block of $\hat{c}$ yields:

$$v \leftarrow F_{k_E}^{-1}(c_1) \oplus c_{l-1} = m_0 \oplus c_0 \oplus c_{l-1}.$$

If the last byte of v is 15, then the last block will be treated as a padding block and removed. The remaining string is valid and will decrypt properly. If the last byte of v is not 15, then most likely the response to the decryption query will be $\perp$. If the response is not $\perp$ then the attacker leans that the last byte of $m_0 = 15 \oplus c_0 \oplus c_{l-1}$.

If the attacker can get an encryption of m using a different IV, then it gets another chance to learn the last byte of m_0. By doing this with enough different IVs an attacker will eventually learn the last byte.

Now imagine the adversary could request an encryption of m shifted one byte to the right to obtain a new ciphertext, they could then perform this same attack again to learn the value of the penultimate byte of m_0. This could be repeated until all bytes of m_0 are revealed.

This attack worked as a complete break of SSL 3.0! Consider a web browser and a victim server, communicating using SSL 3.0. The browser and the server have a shared secret cookie, and the browser embeds this cookie in every request that it sends to the server. Requests from the browser take the form "GET **path** cookie: **cookie**". The attacker aims to recover the secret cookie. The attacker gets the browser to issue a request for the resource $/AA$ at the server, the browser then sends the (encrypted) message "GET /AA cookie: **cookie**". The attacker then edits the ciphertext as described above if there is not an error then it learns one byte of the cookie (and repeats until this happens). The attacker then wants to get the message shifted by one byte, and so gets the browser to request the resource $/AAA$, which then sends the encrypted message "GET /AAA cookie: **cookie**" (i.e. shifts it one byte to the right).

Encrypt-then-authenticate This approach is secure! The intuition behind this is that an adversary will be unable to generate any valid ciphertext $\langle c, t \rangle$ that it did not receive from the encryption oracle (as it would not be able to generate the tag). It also renders the decryption oracle useless, as it either generated the ciphertext from the encryption oracle (and as such knows the decryption) or it will receive an error. Note that the tag is verified before decryption takes place; thus, errors during decryption cannot leak anything about the plaintext, in contrast to the padding-oracle attack we saw against the authenticate-then-encrypt approach.

14.2 Secure Communication Sessions

We briefly describe the application of authenticated encryption to the setting of two parties who wish to communicate "securely"—namely, with joint secrecy and integrity—over the course of a communication session. Where a communication session is a period of time during which the communicating parties maintain state.

Each party maintains two counters $ctr_{A,B}$ and $ctr_{B,A}$ that start at 0 and keep track of the number of messages sent from A to B or (or B to A). A bit $b_{A,B}$ is set to 0 and a bit $b_{B,A}$ set to 1 (or swapped if B is lexicographically smaller than A).

When A wants to transmit a message m to B, A computes $c \leftarrow \text{ENC}_k(b_{A,B}||ctr_{A,B}||m)$, sends c and increments $ctr_{A,B}$. Upon receiving c, B decrypts, parses the decrypted message and checks that the values of the bit and the counter are correct.

14.3 Further reading

For more information on the POODLE attack, see `https://security.googleblog.com/2014/10/this-poodle-bites-exploiting-ssl-30.html`. For further reading see [4] section 9 and [8] section 5.2.

Lecture 15

Password Hashing and identification

Passwords are ubiquitous with modern day security, and often form the first part of any security protocol. When a user logs into a system, they have to provide their username and their password, which the system can then check. Clearly the system can't store the passwords in plaintext, as then if the server is compromised then so are the passwords. The solution? Hash the passwords and store them!

15.1 Password Hashing

The storage and hashing of passwords has improved a lot over recent decades, we'll look at a few different methods and why they have been superseded.

Version 1 - the naive approach The naive password hashing solution is to store a table of user ids and hashed passwords, i.e. each row of the table contains $id_i, H(pw_i)$. Whilst this might be good if the passwords were random, in reality they are not. For example, for 2024 the most common passwords in the UK (according to NordPass) were:

1. password
2. qwerty123
3. qwerty1
4. 123456
5. liverpool
6. 123456789
7. password1
8. qwerty
9. liverpool1
10. arsenal

About 10% of people use a password in the top 25 most commonly used passwords. This suggests that if we had 100 users, then if an attacker made 25,00 guesses there's a good chance that they'd manage to log in to at least one account. This style of attack, where an attacker tries every password from a list of common passwords is known as a dictionary attack - as we often refer to the list of likely passwords as a dictionary $\mathcal{D}$. A common defense against online dictionary attacks is for a server to double it's response time every other time there's a failed login from a specific IP address - however an attacker could use multiple machines (e.g. a botnet) with multiple IP addresses. There are more

non-cryptographic methods for trying to stop this style of attack, however there is one big issue - what happens if the table is leaked?

It is fairly common for these tables to be leaked, and then an attacker has a list of the *id*s and the corresponding password hashes. This makes it even easier for an attacker to launch an attack - with the above example, rather than having to make 25 guesses per user, the attacker can hash each of the 25 most common passwords and then compare each one with each hash in the table.

Offline dictionary attacks are made worse by preprocessing - where attackers use a Space/Time tradeoff before even getting the file. Let $\mathcal{D}$ be the set of likely passwords, given an attacker that has access to S space and can make T queries, then the probability of an attacker finding a preimage of the hash of a given user's password is $\mathbf{Adv}_H^{\text{PREIMAGE}}(S,T) \leq \frac{ST}{|D|} + \frac{T}{|D|}$.

Version 2 - salt. To reduce the impact of preprocessing attacks, and to ensure that an attacker can't just hash the string "password" and check it against every user very quickly, most systems use salting. Salting works by generating a random string, called a salt, when registering a password. Every user is assigned a fresh, random salt- normally a 128-bit string. The salt is hashed with the password, and is stored along with the hash - so each row in the password table now looks like $id_i, salt_i, H(pw_i, salt_i)$.

The reduces the impact of preprocessing, as now the preprocessing can't just cover the set of likely passwords, it has to cover every possible salt with every possible password. Let $\mathcal{S}$ be the set of possible salts, then the probability of an attacker finding a preimage for a specific user is now $\mathbf{Adv}_H^{\text{PREIMAGE}}(S,T) \leq \frac{ST}{|S|\cdot|D|} + \frac{T}{|D|}$.

Whilst salting reduces the impact of preprocessing, we still have the standard offline dictionary attack.

Version 3 - pepper As well as having public salts, often secret salts are used - referred to as pepper. Peppering works by choosing a random pepper from a small salt space (say 12 bits), and then hashing the password with the salt and the pepper but still only storing the salt. Our table now looks like $id_i, salt_i, H(pw_i, salt_i, pepper_i)$. As the pepper isn't stored, both the server - and the attacker - have to try every pepper, for 12 bits this slows everything down by a factor of 4096.

However this isn't enough - most modern GPUs have a hashrate of around 1GH/s (1 billion SHA256 hashes a second). haveibeenpwned has a list of over 613 million leaked passwords with approximately 50-60% of all considered passwords being in this list. Even checking all 4096 peppers for every password on the list would take 40 minutes on a GPU.

15.1.1 PBKDF (Password Based Key Derivation Function) - Slowing down attacks

The speed of password hashing is what makes an offline dictionary attack possible, the question is - can we use a slower hash function?

A naive approach is to hash the password (and salt and pepper) multiple times. For example if hash the password 10,000 times, then is slows down the computation 10,000 times. Whilst this might not make much different to a user, it would make a huge difference to an attacker. It does (unfortunately) come with some other issues - one of which is that iterating H d times (i.e. computing $H^d(x)$) is about d times easier to invert than H.

PBKDF2 A commonly used method to construct a slow hash function from a standard hash function H is PBKDF2, which has a sliding computation cost d.

PBKDF2 works as follows:

- $x_0 \leftarrow salt$
- for $i = 1$ to d
  - $x_i \leftarrow H(pw, x_{i-1})$
- return $x_1 \oplus x_2 \oplus \cdots \oplus x_d$

When PBKDF2 was first proposed (2000) the recommended number of iterations was 1000, now it is over 600,000 - so clearly as hardware has sped up, the impact of this has been lessened. Is there anything we can do to improve this?

15.1.2 MHFs

Rather than increasing the amount of iterations of SHA256, there is another method of constructing slow hash functions - using memory hard functions i.e. a construction that requires a lot of memory for it to be computed efficiently.

GPUS + Bitcoin miners Whilst GPUs have a hashrate of 1GH/s, dedicated hardware (such as antminers) have hashrates of well over 1TH/s. Due to the compact hardware implementation of SHA256, it is possible to make dedicated hardware (ASICs or FPGAs) that can perform many computations in parallel. The bitcoin network alone currently has a hashrate of 772.24EH/s (27/01/25), we need some way of stopping this from being leveraged against password hashing.

We can construct a function that requires a large amount of memory - called a memory hard function. Memory takes up a large amount of space on a chip, and increases the monetary cost of producing such chips substantially.

Scrypt Scrypt was one of the first memory hard functions to be proposed and is widely used - although is not formally recognized as a standard. Given a hash function $H : \{0,1\}^n \to \{0,1\}^n$ and a difficulty d, Scrypt will hash m as follows:

1. $x_0 \leftarrow m$
2. for $i = 1$ to d
   (a) $x_i \leftarrow H(x_{i-1})$
3. $y_0 \leftarrow x_d$
4. for $i = 1$ to d
   (a) $j \leftarrow y_{i-1} \bmod (d+1)$
   (b) $y_i \leftarrow H(y_{i-1} \oplus x_j)$
5. return y_d

For each y_i to be calculated, the attacker is reliant on having either saved x_j in memory or recomputing x_j. There are some tradeoffs that can be done to reduce the amount of memory, but this increases the amount of time required to compute it. In fact it has been shown that for an attacker that uses S memory (where S is at most dn) will still have to make at least $\frac{d^2n}{S}$ hash queries.

Scrypt is often used as a PBKDF, in which case ScryptPBKDF is defined as follows,

1. $m \leftarrow \text{PBKDF2}(pw, salt, 1)$
2. $y \leftarrow \text{Scrypt}(y)$
3. return $\text{PBKDF2}(pw, y, 1)$

dMHF vs iMHF The memory access patter of Scrypt is dependent on the input, and so is referred to as a data-dependent memory hard function (dMHF) - unfortunately this can open it up to some other attacks, such as side-channel attacks. However it is possible to have data-independent memory hard functions (iMHF), where the memory access pattern is completely independent of the input. One such iMHF is Argon2i, whih won the Password Hashing Competition in 2015. The Password Hashing Competition was an independent competition in the style of NIST's standardisation process to decide on AES and SHA-3.

15.2 Multi-factor authentication

Another method of reducing the impact from passwords being leaked, is to use some extra layer of verification of a the user - this is known as multi-factor authentication(MFA).

One time passwords A common method of MFA is to get the user to enter a one time password, typically in the form of a 6 digit code. This is often distributed in the form of a key fob, or an authenticator app. These codes are often generated and verified using hash functions!

Let sk (secret key) represent the key that the user has, and vk (verification key) represent the key the server has. A simple one-time password protocol can be defined as follows:

- Choose a random k, and output $sk = (k, 0)$ and $vk = (k, 0)$ and distribute accordingly.
- When a user is asked to verify, they compute $r = H(sk)$, where $sk = (k, i)$, increment i and return r.
- The server can then check if $r = H(vk)$, since $vk = (k, i)$, and if $r = H(vk)$ output accept and set $vk = (k, i + 1)$.

This has some issues, such as having to ensure that i is consistent across both user and server and if vk is ever leaked from the server, this is no longer secure.

S/Key S/Key is a OTP where leaking the verification key does not lead to the system being insecure. S/Key is defined as follows:

- Choose a random k, output $sk = (k, n)$ and $vk = H^{n+1}(k)$, and distribute accordingly.
- When a user is asked to verify, when $sk = (k, i)$, they compute $r = H^i(k)$, set $sk = (k, i - 1)$ and return r.
- The server can then check $H(r) = vk$, and if $H(r) = vk$ output accept, and set $vk = r$.

This protocol can be edited to remove the consistency issue, but decrementing i every 30 seconds. The server then has to record the time of the last verification, and compute $H^j(r)$ where j is the time between the last verification and the current time, divided by 30.

15.3 Further reading

For more information on how the low entropy of passwords is taken into account in larger protocols - especially with multi-instance security, see [1, 6]. For a more modern version of S/Key, called T/Key see [9]. For further info see chapter 18 in Boneh-Shoup [4].

Lecture 16

KDFs

For most of symmetric key cryptography, we have assumed that we have access to some random key, without considering how it is generated. We now consider this problem!

The key derivation problem is, roughly, is: we start with some secret data, and we want to convert it into an n-bit string that we can use as the key to some cryptographic primitive, like AES. The secret data may be random in some sense, but may not be uniformly distributed, or may not be an n-bit string. How do we get from a secret s to a cryptographic key t? Unsurprisingly, the solution involves hashing, in which we take some hash function H (or some function built from H) and compute $t = H(s)$.

What could the secret be? We'll cover a couple of examples.

- A password. We've already discussed that passwords typically are not that random, but even if we assume we do have a password that has 128 bits of entropy, it's not a uniformly random string. A password comprised of ASCII characters for example will have redundancy since only a portion of the characters are printable.
- A secret taken from some source of high entropy, but where the bitstring representation is many times larger than the amount of entropy. Examples include photos of lava lamps, logs of system events from a computer, photos of pendulum movements.
- A previously secure key what part of it has been compromised, e.g. some of the bits were leaked to the adversary.
- A value from a public-key key exchange e.g. a, where the value $g^a \mod p$ for some generator g and prime p is public. Here a may not be chosen as a uniform bit string, and there is some leakage.

16.1 KDFs

Before start by looking more formally at the security notion for KDF security, we consider a Guessing game. In the Guessing game a secret s is sampled from a distribution P over the set $\mathcal{S}$. Some side information z about the secret is also known (this could be some of the bits of s, or it could be the Hamming weight of s, or some other leakage). An adversary is given the leakage z and has to guess some s', and they win if $s' = s$.

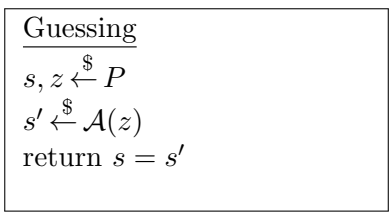


Figure 16.1: The Guessing Game

Now consider the case where we use a hash function ($H : \mathcal{S} \to \{0,1\}^n$)to derive the key t from s. We want t to look random, to formalize this we use a notion of indistinguishability. Formally, if a secret s is sampled from P with leakage z, and t is chosen uniformly at random from $\{0,1\}^n$, we want the two probability distributions $(z, H(s))$ and (z,t) to be indistinguishable.

Distinguishing
$b \xleftarrow{\$} \{0,1\}$
$s, z \xleftarrow{\$} P$
if $b == 0$
$t \leftarrow H(s)$
else
$t \xleftarrow{\$} \{0,1\}^n$
$b' \xleftarrow{\$} \mathcal{A}(z,t)$

Figure 16.2: The Distinguishing Game

Let W_b be the event that $\mathcal{A}$ outputs 1 for each value of b, then $\mathcal{A}$'s advantage is

$$\mathbf{Adv}^{\text{Dist}}(\mathcal{A}) = |\Pr[W_0] - \Pr[W_1]| \ .$$

16.1.1 KDFs and random oracles

We now consider the key derivation problem, where we are interested in how well an adversary can distinguish between $(z, H(s))$ from (z,t) when t is random. We want to argue that if we model a hash function as a random oracle, then the advantage of an adversary at distinguishing between these is not much more than the adversary's advantage at being able to guess s from the side information z.

We start by extending the Guess game to the ListGuess game, in which rather than the adversary returning s', and winning if $s' = s$, the adversary returns a list $s'_1, \ldots, s'_q$ and wins if $s \in s'_1, \ldots, s'_q$.

We have that for all adversary $\mathcal{A}$ against ListGuess, we can construct an adversary B against Guess such that

$$\mathbf{Adv}^{\text{ListGuess}}(\mathcal{A}) \leq q\mathbf{Adv}^{\text{Guess}}(B) \ .$$

Where B runs the adversary $\mathcal{A}$ and then picks one of the q values at random.

Theorem 16.1. *For any adversary $\mathcal{A}$ trying to distinguish between $(z, H(s))$ and (z,t) when t is random, there is an adversary $\mathcal{B}$ that wins the ListGuesssing game, i.e.*

$$\mathbf{Adv}^{Dist}(\mathcal{A}) \leq \mathbf{Adv}^{ListGuess}(\mathcal{B})$$

Proof. We start by writing a Game 0 that is the same as the $b = 0$ game for distinguishing, but with a random oracle that is built up on the fly.

Game0
$Map : \mathcal{S} \to \{0,1\}^n$
$s, z \xleftarrow{\$} P$
$t \xleftarrow{\$} \{0,1\}^n$
$^*Map[s] \leftarrow t$
$b' \xleftarrow{\$} \mathcal{A}^{\mathcal{O}}(z,t)$

$\mathcal{O}(x)$
if $Map[x] = \perp$
$Map[x] \xleftarrow{\$} \{0,1\}^n$
return $Map[x]$

We now move to a second game, Game 1, where we delete the line marked $*$.This game is equivalent to the $b = 1$ game for distinguishing. Both games result in the same outcome, unless the adversary $\mathcal{A}$

in Game 1 makes the query $\mathcal{O}(s)$. Our adversary B runs $\mathcal{A}$, and at the end outputs the list of queries that $\mathcal{A}$ made. B is able to do this without knowledge of s besides z. Thus

$$\mathbf{Adv}^{\text{Dist}}(\mathcal{A}) = |\Pr[W_0] - \Pr[W_1]| \leq \mathbf{Adv}^{\text{ListGuess}}(\mathcal{B})$$

□

16.2 KDFs in practice

There are a number of KDFs that get used in practice, one we've already seen - PBKDF2. Another is HKDF, which is a KDF built from HMAC. HKDF has two stages, one that extracts the randomness from the input, and the other that expands this to be the required length. In the extract stage for a salt sa and a random value s we compute $t \leftarrow \text{HMAC}(sa, s)$. If sa is empty then by modelling HMAC as a random oracle, we have that if s is hard to guess then t is indistinguishable from a random string. If sa is not empty then we can make use of the leftover hash lemma to get the same result without requiring the random oracle (in fact the LHL makes use of hash functions with slightly weaker properties than those we've looked at). In the expand stage, if we need q times the output length of HMAC we compute the expansion as $z_1||\ldots||z_q$, where z_0 is the empty string and $z_i \leftarrow HMAC(t, z_{i-1}||i)$.

16.3 Sources of entropy

Cloudflare famously has different sources of entropy in each of it's three main offices. The first (and most well known) is a wall of lava lamps. The flow of lava in a lava lamp is highly unpredictable, and this is exploited by having a camera pointing at the wall of lava lamps that takes photos of the wall. Each image is then used as the 'secret' that is fed into the KDF, where the entropy between each image is associated with the location of the globs of lava in each lamp.

In Cloudflare's London office, they have a camera pointing at a double pendulum - which is a chaotic system. Here the entropy comes from the differences in location of each pendulum between images being taken.

Finally in the Singapore office they have a pellet of Uranium which they measure the radioactive decay of.

16.4 Further reading

For more information on how we extract randomness from multiple sources of entropy, see [5] `https://eprint.iacr.org/2025/1258` .

Lecture 17

Merkle Trees

We've considered how to guarantee the integrity of data, by storing the hash of the data. But what about large files? Computing the hash of a large file is very time consuming, both techniques we've considered for constructing hash functions (Merkle-Damgard and Sponge) are inherently sequential. If we can't open/run the file until the entire hash is checked, this causes a huge lag.

Naive solution Rather than hashing the entire file, we split the file into blocks and hash each block individually. Whilst this would allow us to verify blocks individually, it would require a large amount of storage.

17.1 Merkle trees

To restate the problem, we have a vector of n items $T = (x_1, \ldots, x_n)$, and we wish to compute a short hash y of this vector. Later, we are presented with the hash y and a pair (i, x) where $1 \leq i \leq n$. We need to validate that x is the item at position i in T.

A solution to this problem makes use of a clever data structure called a Merkle tree. The resulting hash function H is called a Merkle tree hash.

The Merkle tree hash uses a collision resistant hash function h and constructs H as follows. For simplicity, let's assume that n is a power of two (if not, one can pad with dummy elements to the closest power of two).

1. Pad T with dummy elements until $|T| = n$ is a power of 2.
2. For $i = 1$ to n: compute $y_i \leftarrow h(x_i)$.
3. For $i = 1$ to $n - 1$: compute $y_{i+n} \leftarrow h(y_{2i-1}, y_{2i})$.
4. output y_{2n-1}.

Proving membership Given a value $y \leftarrow H(x_1, \ldots, x_n)$, it is quite easy to prove that some x is an element of T in a particular position. For example let $n = 8$, to prove that x_3 is a member of T one can provide intermediate hashes y_4, y_9, y_{14}. This can be used to verify by computing $\hat{y}_3 \leftarrow h(x), \hat{y}_{10} \leftarrow h(\hat{y}_3, y_h), \hat{y}_{13} \leftarrow h(y_9, \hat{y}_{10}), \hat{y}_{15} \leftarrow h(\hat{y}_{13}, y_{14})$. If $y = \hat{y}_{15}$, then x must be x_3.

More generally, to prove that element x is the element in position i of T, one outputs all the intermediate hashes that are the siblings of nodes on the path from the leaf number i to the root of the tree. The proof requires exactly $\log_2(n)$ elements, and a verifier can use the values in this proof to recompute the Merkle hash of T. It does so by computing hashes, starting at leaf number i, and working its way up to the root. It accepts that x is authentic (i.e., that $x = x_i$) if the final computed Merkle hash matches the hash value y.

Proving membership of multiple elements Suppose we want to prove that all elements of a list L are in T. We could provide a Merkle proof for every element of L - giving a total of $|L| \log_2(n)$ elements in the proof. However many of them may overlap, in fact we can show that in the worst case, $|L| \log_2 n/|L|$ elements are needed.

17.2 Sorted Merkle tree

As well as using a Merkle tree for hashing one large file, another use case is to consider T distinct secret elements, with the aim of either proving membership or non-membership.

Since we're primarily interested in proving (non)membership of an element, we can order the elements first. A sorted Merkle tree hash, is a modified tree hash where the leaves of the tree are in some lexicographical order.

Proving non-membership Given some $x \notin T$, we want to produce a proof that x is not in T. To produce the proof, we first find two adjacent leaves x_i and x_{i+1}, such that $x_i < x < x_{i+1}$. We then produce a proof that x_i is in position i and that x_{i+1} is in position $i+1$. Since x_i and x_{i+1} are adjacent nodes, this proves that x is not in T, as if x was in T it must be between those two leaves. In the worst case, proof of non-membership contains $2 \log_2(n/2)$ elements.

17.3 Authenticated data-structures

A Merkle tree isn example of a more generalized structure known as an authenticated data structure. An authenticated data structure is used to compute a short hash of a sequence T, so that later one can prove properties of T with respect to this short hash. Merkle trees allow for proving (non)membership, however some other authenticated data structures allow for efficient insertions and deletions.

Formally an authenticated data structure scheme $\Pi = (H, P, V)$ is a tuple of three efficient deterministic algorithms:

H is an algorithm that is invoked as $y \leftarrow H(T)$, where $T = (x_1, \ldots, x_n)$.

P is an algorithm that is invoked as $\pi \leftarrow P(i, x, T)$, where $1 \leq i \leq n$, and π is a proof that $x = x_i$.

V is an algorithm that is invoked as $V(i, x, y, \pi)$ and outputs accept or reject.

Where for all $T = (x_1, \ldots, x_n)$, and all $1 \leq i \leq n$, we have that $V(i, x_i, H(T), P(i, x_i, T)) = \text{accept}$. It is clear to see that the Merkle tree scheme meets this definition.

We define the security game for an authenticated data structure as follows.

ADS Sec

$(y, i, (x, \pi), (x', \pi')) \xleftarrow{\$} \mathcal{A}^h()$

return $x \neq x' \wedge V(i, x, y, \pi) = V(i, x', y, \pi') = \text{accept}$

Figure 17.1: The authenticated data structure security game

17.4 Further reading

For further reading see [4] section 8.9 and [8] section 6.6.2.

Bibliography

[1] Mihir Bellare, Thomas Ristenpart, and Stefano Tessaro. Multi-instance security and its application to password-based cryptography. pages 312–329, 2012.

[2] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. pages 62–73, 1993.

[3] Mihir Bellare and Phillip Rogaway. Code-based game-playing proofs and the security of triple encryption. Cryptology ePrint Archive, Report 2004/331, 2004.

[4] Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography*. 0.6 edition, 2023.

[5] Sandro Coretti, Pooya Farshim, Patrick Harasser, and Karl Southern. Multi-source randomness extraction and generation in the random-oracle model. pages 10:1–10:23, 2025.

[6] Pooya Farshim and Stefano Tessaro. Password hashing and preprocessing. pages 64–91, 2021.

[7] Martin Hellman. A cryptanalytic time-memory trade-off. *IEEE transactions on Information Theory*, 26(4):401–406, 1980.

[8] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. CRC Press, third edition, 2021.

[9] Dmitry Kogan, Nathan Manohar, and Dan Boneh. T/key: Second-factor authentication from secure hash chains. pages 983–999, 2017.

[10] Ralph C. Merkle. A certified digital signature. pages 218–238, 1990.

[11] Phillip Rogaway and Thomas Shrimpton. Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance. pages 371–388, 2004.

[12] Nigel P. Smart. *Cryptography Made Simple*. 2016.